Project Report: Ticket Flash Sale

CS6550 - Building Scalable Distributed Systems

Harrison Pham, Chia-hsiang Hsu Tai

Video Link: *https://www.youtube.com/watch?v=-NbNDENvodI*

Github Repository: *https://github.com/tchiahsu/flash-sale-system*

Introduction

Flash sale systems, such as those used by ticketing platforms, must handle extreme concurrency while maintaining strict correctness guarantees. When hundreds or thousands of users attempt to purchase a limited number of items simultaneously, systems can fail in three critical ways: overselling tickets by violating inventory constraints, creating duplicate orders where the same ticket is sold multiple times, and losing transactions because a successful sale is never recorded. Each of these failure modes has a direct impact on real-world vendors, buyers and event organizers.

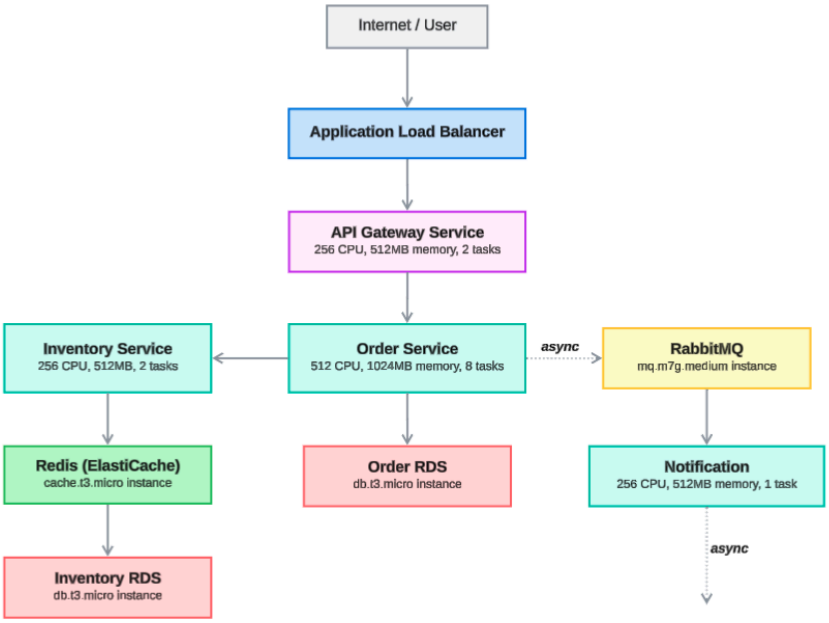
The problem matters because correctness and performance are both non-negotiable in a ticket flash sale context. A system that is fast but inconsistent will oversell and cause disputes. A system that is fast but inconsistent will oversell and cause disputes. A system that is correct but slow will time out under load and fail to process valid transactions. Our project builds and stress-tests a distributed flash sale system to evaluate how specific architectural decisions affect performance and correctness, and to identify how the system behaves under realistic and extreme loads

Overview

This project builds a distributed ticket flash sale system using a microservices architecture deployed on AWS ECS Fargate, consisting of four services, API Gateway, Order Service, Inventory Service, and a Notification Service.

When a user sends a purchase request, the API Gateway receives it and forwards it to the Order Service. The Order service acts as the coordinator for the purchase flow. It makes a synchronous HTTP call to the Inventory Service to attempt a ticket reservation, waiting for a response before proceeding. The Inventory Service performs an atomic decrement on the ticket count, using either Redis or a Postgres atomic update depending on the backend configuration and returns either success or failure. If the reservation succeeds, the Order service writes a confirmed order to its PostgreSQL database and returns a confirmation to the user. If it fails, the user receives a sold-out response immediately.

After confirming an order, the Order Service publishes a message to RabbtiMQ. The Notification Service consumes these messages asynchronously and records a notification for each confirmed order. Because this step happens outside the purchase path, it never blocks or slows down the user's request. This was a design choice to decouple the notification delivery from purchase correctness.



System Architecture Overview

Project Management

Project Progression

Our initial proposal described a four-service architecture: an API Gateway, Inventory Service, Order Service, and Notification Service. The original experiment plan covered horizontal scaling, fault tolerance, and a Redis vs. PostgreSQL inventory comparison. At this stage the design was intentionally high-level. We had not yet decided how services would communicate, where state would live, or how the async pipeline would work.

Several significant decisions were made as implementation began:

- **Service communication model.** The initial proposal left communication unspecified. After working through the purchase flow, we settled on a hybrid synchronous/asynchronous design. The critical path, Gateway → Order Service → Inventory Service, is synchronous because users need an immediate answer on whether they secured a ticket. The notification side effect is asynchronous through RabbitMQ because it does not affect purchase correctness and should not block the user.
- **Role clarification between Order and Inventory.** Early on it was unclear which service should orchestrate the purchase flow. We determined that the Order Service should act as

the coordinator. It calls Inventory synchronously to reserve a ticket, writes the order record to PostgreSQL, and publishes to RabbitMQ. The Inventory Service is responsible for tracking the number of tickets and protecting the ticket count during a decrement. This separation allowed us to independently scale and swap each service during experiments.

- **Experiment 1 redesign.** The original Experiment 1 targeted the Inventory Service for horizontal scaling. After analysis we thought the Order Service would be the actual bottleneck, as it does the most work per request. Experiment 1 was revised to scale the Order Service. Results from Experiment 1 show that the bottleneck lies downstream of the Order Service, which directly motivates the need for the system to perform Experiment 3.

- **Experiment 3 redesign**. The original Experiment 3 was comparing Redis-only to Postgres-only which didn't make sense and was highlighted in one of our mock interviews. The experiment was modified to compare Redis+Postgres to Postgres-only to see if the use of Redis to make atomic reservations was significantly better than Postgres row-level lock contention under high concurrency.

- **Addition of RabbitMQ resilience experiment.** The original proposal had three experiments. A fourth experiment was added to test RabbitMQ behavior under stress. This was motivated by our desire to learn RabbitMQ and the observation that most projects treat the message queue as infrastructure that just works. We wanted to explicitly test what happens when it doesn't.

Problems Encountered

RDS subnet configuration. Making RDS publicly accessible for the consistency harness required both setting `publicly_accessible = true` and moving the subnet group to public subnets. Terraform could not modify the subnet group on a running RDS instance, requiring the instances to be destroyed and recreated.

RabbitMQ reconnection. Both the Order Service and Notification Service initially had no reconnection logic. If the broker went down, the services would silently fail without recovering. This was discovered during Experiment 4a testing and fixed by implementing reconnection loops in both services before running the experiment.

Flawed consistency check. The original consistency check script sent 200 fresh purchase requests as part of the check, this meant that running the consistency check after locust would already produce 200 failures making the results meaningless. The script was modified and rewritten to test for correctness more accurately.

Redis backend not switching. For Experiment 3, we compared Postgres-only vs Redis+Postgres strategies. The terraform file was set up so that changing the `INVENTORY_BACKEND` variable between `"redis-postgres"` and `"postgres"` would make the change. However, we found

that the task definitions in ECS would not change despite running `terraform apply` and `deploy.sh`. In order to get it to change, we manually forced the update through the AWS CLI.

Task Ownership

Harrison Pham	Chia-hsiang Hsu Tai
Implement Inventory Service	Implement API Gateway
Implement/Integrate Redis on AWS	Implement Order Service
Implement/Integrate PostgreSQL RDS	Implement Notification Service
Implement/Integrate RabbitMQ	Experiment 1 Initial Testing
Experiment 1 Validation and Execution	Experiment 2 Execution
Experiment 4 Execution	Experiment 3 Execution
Joint Contributions	
Terraform and Infrastructure Setup	
Consistency Verification Script	
Final Report Write Up	
Prepare Project Presentation	

Challenges

Identifying the Bottleneck. It was not obvious initially whether the inventory service, order DB, or API gateway would be the bottleneck. A trial run was needed to observe the CloudWatch metrics in real time which showed that we had to change Experiment 1 to properly test the bottleneck. The trial revealed that the inventory service was the bottleneck, which allowed us to narrow down our experiments.

Determining Load Parameters. We wanted to test the system under load, so we had to experiment with how many tickets to have, the number of concurrent users, and the length of the load test. For experiment 2 for example, we had 200 users, 100 tickets configuration, but that wasn't enough for Experiment 3, so we had to significantly change the setup to 1,000 users and 10,000 tickets to actually see results. We did two rounds of testing to get the right setup before conducting the actual experiment for Experiment 3.

Redis activity from idle connections. Because the inventory service always initializes a Redis client regardless of whether we use it or not, the `CurrConnections` metric in CloudWatch showed 4-5 connections even during the Postgres-Only run, which made us think our

architecture was incorrect. This led to us reviewing the backend with Claude Code to understand if the implementation was correct.

Experiments and Results

Experiment 1: Horizontal Scaling Under Drop Traffic

Purpose

This experiment evaluated whether horizontally scaling the Order Service improves system throughput and latency during a flash sale. Four configurations were tested: 1, 2, 4, and 8 ECS tasks, each under a fixed Locust load of 100 users, a spawn rate of 100, a 1-minute duration, and 100 available tickets. This setup simulated a flash sale spike where users arrive at the same time competing for a limited inventory of tickets. The experiment also included an auto-scaling test, running for 11 minutes, configured with a 20% CPU threshold, a 30-second scale-out cooldown, and a maximum of 8 tasks, to observe how long ECS took to detect elevated load and provision new tasks. The goal was to determine whether adding additional tasks improved request handling and whether auto-scaling could be relied on as a strategy for sudden demand.

Tradeoffs Explored

Horizontal scaling is often presented as a straightforward solution to handling increased load. Add more replicas and the system handles more traffic. However, every additional Order Service replica still routes synchronous calls through the same Inventory Service and writes to the same Orders PostgreSQL database, meaning the bottleneck may shift downstream regardless of how many replicas are added. The tradeoff explored is whether scaling the compute tier actually improves performance, or whether the bottleneck lies in the shared resources beneath it.

The auto-scaling portion of this experiment explores a second tradeoff: reactive scaling offers operational simplicity, but introduces a reaction delay that may make it ineffective for flash sales where the entire traffic spike arrives instantaneously.

Limitations of the Approach

This experiment had several limitations that affect how broadly its conclusions can be applied. The fixed replica tests ran for only 1 minute each, which captures steady-state performance but does not reflect the true spike behavior of a flash sale where load arrives instantaneously and subsides quickly. The auto-scaling policy also had to be configured at a 20% CPU threshold, far below the typical 70%, because the Order Service never consumed significant CPU under load. This made it difficult to use auto-scaling in a realistic way. Together, these limitations suggest that the results reflect controlled testing conditions rather than the unpredictable nature of a real flash sale.

Results

Fixed Scaling

Number of Tasks	RPS	Median (ms)	P95 (ms)	P99 (ms)	Average (ms)
1	181.7	310	1500	1800	491.54
2	165.6	400	1400	1700	544.8
4	170.8	420	1400	1600	555.57
8	167.5	570	1200	1400	576.13

Autoscaling Reaction Time

Time (min)	Number of Tasks	Avg CPU
0	1	-
2	1	~40%
4	1	~40%
6	2	~20%
8	4	~10%
10	4	~10%

Analysis

The results from the fixed replica tests show that adding more Order Service tasks had no meaningful impact on throughput. RPS remained roughly constant across all four configurations, ranging from 165.6 to 181.7, with no clear upward trend as replicas increased. Median latency gradually worsened from 310ms at 1 task to 570ms at 8 tasks, while p95 and p99 showed slight improvement. These changes are minor and do not indicate a meaningful performance gain from scaling. This provides evidence that the bottleneck was never in the Order Service, and that scaling it further only increases contention on the shared downstream resources it depends on: the Inventory Service and the Orders PostgreSQL database. This directly addresses the tradeoff identified in this experiment: scaling compute capacity did not improve performance because the bottleneck lies elsewhere in the system.

The auto-scaling results show that CPU utilization peaked at approximately 40% during the test, after several minutes of sustained load. ECS took 6 minutes to provision a second task and 8

minutes to reach 4 tasks, reflecting the reaction delay found in CPU-based auto-scaling policies. This evidence suggests that the auto-scaling responds too slowly relative to the speed at which flash sale demand arrives, supporting the tradeoff identified between operational simplicity and reaction time.

As mentioned earlier, there are limitations to this analysis. Each fixed replica test ran for only 1 minute, which may not be long enough to produce stable results. Similarly, the auto-scaling test reflects only one scaling policy configuration, and different threshold or cooldown values may have produced different results.

Conclusions

The results of this experiment demonstrate that horizontally scaling the Order Service had no meaningful effect on throughput or latency, as the bottleneck lies downstream in the shared Inventory Service and Orders PostgreSQL database rather than in the compute tier itself. Scaling the wrong tier wastes resources without improving performance, and any future optimization effort should focus on the downstream dependencies instead. The auto-scaling results further demonstrate that reactive scaling is not a viable strategy for flash sales. Even with a 30-second scale-out cooldown, ECS took 6 to 8 minutes to provision additional tasks, far too slow to absorb a demand spike that arrives instantaneously. For a flash sale system, tasks should be scaled out in advance of the event rather than relying on auto-scaling to react in real time.

Experiment 2: Fault Tolerance during Flash Crowd Sale

Purpose

The purpose of this experiment was to evaluate the fault tolerance of our flash sale system under high-traffic conditions when a partial failure occurs to one of the crucial services. Specifically, the experiment simulated a scenario where one of the two running Inventory Service ECS tasks was manually terminated during the high-traffic conditions where 200 users were actively sending purchase requests. The goal was to understand how much request loss occurred during the failure window, how long the system took to recover, and whether data correctness was preserved throughout the entire incident.

This experiment was designed to model the reality that distributed systems operating under high traffic are especially vulnerable to partial failures. Flash sales are the worst-case scenario since there is a sudden spike in demand for the system. Understanding how our system would react when a critical component fails is important to evaluate our system.

Tradeoffs Explored

The core tradeoff in this experiment is correctness and availability under partial failure. In any distributed system, this is a tradeoff that is important to consider and understand. A system that focuses on availability might continue to process requests even with state data or inconsistent

state, which can lead to overselling or duplicate orders. A system that focuses on correctness will fail all requests the moment it cannot guarantee order will be processed safely, sacrificing availability.

The way we have designed our system, we make decisions to try and find a balance between availability and correctness. When requests come in to purchase a ticket, we use an atomic PostgreSQL `UPDATE` with a `WHERE remaining >= $1` guard, which means that decreasing inventory is safe even under concurrent load. The order service calls the inventory service synchronously before writing to the orders database, meaning if the inventory service is unreachable, the order fails cleanly rather than proceeding without first checking inventory. Experiment 2 tests whether these design choices preserve correctness when one inventory service instance fails in the middle of the flash sale.

Limitations of the Approach

There are several limitations that constrain the conclusion from this experiment:

- **Manual kill timing**. The inventory service task was terminated manually via the AWS ECS console. This means the exact moment of the task kill relative to the Locust timeline is relative, and we identified it by looking at the Locust RPS dip with the CloudWatch CPU spike rather than an actual specific timestamp.

- **Single failure scenario**. The experiment tested only one specific failure mode, a clean task failing. Real-world failures are usually more convoluted, a task might become unresponsive without dying, or returning wrong responses, or failing intermittently. The ALB health check would not immediately detect a hanging task, all of which would result in a different behavior and different results

- **No auto-scaling configured**. The ECS service uses a fixed number of tasks defined in Terraform. This means the experiment cannot evaluate how auto-scaling would impact the recovery time.

- **Short Test Window**. The test ran for approximately 180 seconds. The kill occurred roughly 45 seconds in, which we believe to be enough for the system to recover and reach steady-state. A longer test would provide cleaner separation between baseline, degraded, and recovered phase.

Results

Locust Load Test Configuration

Parameter	Value
Total Users	200
Spawn Rate	50 users
Run Time	180 seconds
Wait Time	0 to 0.1 seconds

Locust Statistics

Metric	Value
Total Requests	21,233
Total Failures	372
Failure Rate	2%
Median Response Time	1,300ms
95th Percentile	4,300ms
99th Percentile	5,000ms
Average Response Time	1,620ms
Min Response Time	25ms
Max Response Time	5,650ms
Peak RPS	~150
End RPS	~113

Analysis

The system demonstrated graceful degradation. The 2% failure rate (372 out of 21,233 requests) during partial infrastructure failure shows the system is fault tolerant. It means the ALB's health check successfully detected the dead task and rerouted traffic to the only surviving instance before failures could accumulate significantly.

The availability vs. correctness tradeoff resolved and leaned more towards correctness. All 5 consistency checks passed. This means that exactly 100 orders were confirmed (initial inventory was 100 tickets), and there were no duplicate order IDs. Every confirmed order had a corresponding notification record. The inventory reconciliation showed that from the initial 100 tickets, 100 were confirmed. Despite partial failure occurring mid-test, the system's atomic reservation and chain of sequential service calls ensured that every successful order was legitimate and every failed request failed cleanly rather than leading to a corrupted state.

The 86% CPU utilization peak on the inventory service is the most concerning. With 256 CPU units per task and only one surviving instance, the service received all the requests from the 200 user load. If the load had been higher, it is likely that the only surviving task would have become overloaded and failed. The orders RDS showed flat database connections and low write latency throughout. However, it also means this experiment does not stress-test the orders database, and so we can't make any assumptions on the behavior of the orders database under high load. The 227 inventory DB connections at peak were all handled by the single surviving task, combined with the 86% CPU utilization that confirms that the instance was operating at the boundary of its capacity but it was able to handle partial failure.

The failure rate never produced any significant spike in failure in the Locust test. The main observable effect of killing the instance was a reduction in throughput from ~150 RPS to ~80 RPS and an increase in 95th percentile latency reaching highs of 4,300ms. This pattern is consistent with what we expected from killing a service, the ALB was able to successfully reroute the traffic to the surviving instance so most requests either succeeded or were queued rather than rejected immediately, because there are more requests, we also expected the latency to significantly increase.

The analysis is constrained by many factors. The experiment was run once, the 180 second run time might not have been sufficient to observe full recovery to steady state, meaning recovery time is not very precise. The failure mode tested is killing a task cleanly, there are other ways in which failure could have been tested like a hanging task or network failure that produce worse results. The experiment was conducted with 200 users, which might not be enough to see the full repercussions of the task dying since production level load is significantly higher.

Conclusions

The experiment provides evidence that this system architecture handles partial failure with correctness intact. The 2% failure rate and perfect consistency check results confirm that our design choices for inventory, orders, and notifications service achieves the desired guarantees under realistic failure conditions.

However, the 86% CPU peak on the surviving inventory task shows that the current configuration has minimal fault tolerance headroom. The system passes the test at 200 users with one of two tasks running, but it does so at near-maximum capacity. Increasing the load or the duration of the failure window would likely push the system into further issues by spiking the

CPU Utilization further. Since auto-scaling is not configured for this test, the consistent traffic spike combined with partial failure would likely risk the system's ability to remain alive.

Experiment 3: Redis vs Database-only Inventory Management

Purpose

The purpose of this experiment was to compare two inventory strategies under high concurrency flash sale conditions, a Postgres-only atomic update versus a Redis and Postgres hybrid approach. The experiment was designed to compare the performance and correctness tradeoffs between the two strategies under the same conditions, and to identify which strategy was better at scale.

Inventory management is one of the most important aspects of a flash sale system. Every purchase requires reservations that have to be fast and correct to prevent timeouts and overselling, which can be detrimental in the real-world.

Tradeoffs Explored

The most important tradeoff that is being made is speed vs consistency. The Redis+Postgres hybrid uses Redis atomic operation to make reservations on the tickets eliminating the need for row lock contention in Postgres. Redis will process the decrement in microseconds, and Postgres will be updated asynchronously which means there is a window of eventual consistency. This means redis is the main source of truth during the test. The flip side is using Postgres-only which is a simple implementation, one database, one operation and a single source of truth. However, under high load all requests wait for the same row lock, serializing execution.

Limitations of the Approach

There are several limitations that constrain the conclusion from this experiment:

- **Async sync is not production-grade**. The Redis+Postgres asynchronous synchronization implementation gets run once per reservation to write to Postgres. Under high load this creates a significant amount of DB writes that can lag behind Redis, which was shown by the failure of Check 4 in the consistency test.
- **Single inventory row**. We only use one row with the count for a single event. In a real flash sale the inventory may be split across multiple rows or partitions, which would change the Postgres contention profile.
- **No connection pooling**. The inventory service connects directly to RDS without a connection pooler, with 1,000 users the raw connection count reached 459, which is really high and might have contributed to Postgres high failure count.
- **Redis Single-point of failure.** The Redis configuration has a single node and no replication, this means that if that single Redis node goes down, the entire inventory system fails completely.

Results

Locust Load Test Configuration

Parameter	Value
Total Users	1,000
Spawn Rate	100 users/second
Run Time	180 seconds
Initial Inventory	10,000 tickets
Wait Time	0 to 0.1 seconds

Locust Statistics

Metric	Postgres Only	Redis + Postgres
Total Requests	23,895	136,622
Failures	19,500	1,256
Failure Rate	82%	1%
Median (ms)	5,000	490
95th Percentile (ms)	5,100	2,600
99th Percentile (ms)	5,200	4,700
Average (ms)	4,678	746
Min (ms)	52	25
Max (ms)	5,384	5,339
Peak RPS	~200	~1,482

CloudWatch Metrics

Metric	Postgres Only	Redis + Postgres
Inventory DB Connections (peak)	459	160
Order DB Connections (peak)	~220	~15
Inventory CPU Peak	78.2%	90.9%
Inventory Write Latency (peak)	4.5ms	3.5ms
Redis CurrConnections	N/A	~45

Analysis

The Postgres-Only RPS shows the system immediately failed once all 1,000 users were active. RPS and failures became nearly identical lines at approximately 200, meaning almost every request was failing. Response times hit 5,000ms immediately and stayed there for the rest of the test. The system never recovered. For Redis + Postgres, the RPS shows a health ramp to approximately 1,200 RPS with some spikes in p95 of 5,000ms during the initial ramp up as all 1,000 users hit simultaneously. After the initial spike the latency settled to 2,000-3,000ms for p95 as inventory depleted and requests started to be unsuccessful. The system handled the entire high load scenario with 1% failure.

One significant finding was that Postgres-only didn't only fail on performance but it also failed the correctness test in the consistency script we ran after. It confirmed 10,362 orders when the inventory was 10,000. This could've happened because under extreme load, the lock contention became unreliable when hundreds of connections were timing out, retrying, and reading stale values. This showed that at scale, Postgres alone is not just slow but unsafe.

On the flip side, Redis + Postgres showed great performance with 136,622 requests, 1% failures, and a median latency of 490ms. The system was healthy and processed 1,482 RPS, 7x higher throughput than Postgres alone. Because Redis is single-threaded, all ticket reservations are serialized and processed in microseconds, eliminating the contention issue that Postgres had, which resulted in better performance and correctness.

Postgres-only inventory connections peaked at 459 while order DB also spiked to 220. Redis+Postgres inventory connections peaked at 160 while orders DB stayed at around 15. This shows that Redis absorbs the ticket reservation load from the rest of the system. The CPU utilization also showed that Redis+Postgres is more CPU intensive despite performing better. This might be because managing the Redis client and the asynchronous updates to Postgres adds compute overhead.

The analysis is limited by the implementation and design choices that impact the results. The async Postgres sync is implemented through goroutines, which is not good for production level systems. The absence of a connection pool means that Postgres-only results represent a

worst-case scenario, and adding one could result in better performance. The system also tested a single ticket count, which in a real-world system usually involves sharding inventory across multiple rows to distribute lock contention.

Conclusions

The experiment showed that the two inventory strategies have different behavior under high concurrency. Postgres-only is simple and correct at low load but fails in performance and correctness with 1,000 concurrent users, with 82% failure rate, 5,000ms median response time, and only 200 RPS. Redis with Postgres is more complex and introduces eventual consistency risk in Postgres but it had 1% failures, 490ms median response time, 1,482 RPS, and zero overselling under the exact same load. The significant improvement while maintaining correctness showed that the Redis + Postgres architecture is the best inventory strategy for a system under high concurrent load.

The most interesting finding is that Postgres oversold by 362 tickets. This is not a performance issue but it's more a correctness issue that would have real consequences in the real-world. Redis did not oversell by a single ticket. The experiment also showed that the speed and atomicity of Redis's operations outperformed Postgres row locking in both performance and correctness. The cost of this advantage is eventual consistency in the Postgres database and increased architectural complexity. For a production flash sale system, handling thousands of concurrent users, Redis-based inventory strategy is faster and the better approach.

Experiment 4: RabbitMQ Resilience Under Load

Purpose

This experiment evaluated the resilience of RabbitMQ as the messaging layer in the flash sale system under two failure scenarios. The first part simulated a broker outage by stopping RabbitMQ mid-sale while 100 users were actively sending purchase requests at a spawn rate of 100, with 100,000 available tickets and 2 Notification Service tasks running. The test ran until the queue came back online, with the goal of understanding how many notification messages were lost during the downtime window and whether the Order Service continued processing orders correctly. The second part introduced a 100ms artificial delay in the Notification Service to simulate a slow consumer, running for 1 minute with 10 users at a spawn rate of 10, with the goal of determining whether RabbitMQ could absorb a large message backlog without dropping messages or degrading Order Service throughput.

Together, these two parts were designed to evaluate whether async messaging via RabbitMQ provides reliable decoupling between the Order Service and the Notification Service under both failure and backpressure conditions.

Tradeoffs Explored

The core tradeoff in this experiment is between availability and guaranteed notification delivery. By design, the Order Service confirms a purchase and returns a successful response to the user regardless of whether the notification message was successfully published to RabbitMQ. If the broker is unavailable, the Order Service logs a warning and moves on rather than blocking or failing the order. This keeps the purchase path highly available but means notifications are delivered at most once, meaning messages published during broker downtime are permanently lost with no retry mechanism.

The backpressure portion of this experiment explores a second tradeoff: decoupling the Notification Service from the Order Service via a message queue allows the Order Service to continue processing orders at full speed without being slowed down by a slow consumer, but this comes at the cost of delayed notification delivery and a growing message backlog if the consumer cannot keep up.

Limitations of the Approach

This experiment had several limitations that affect how broadly its conclusions can be applied. In part 4a, the broker outage was triggered manually and lasted a controlled duration, meaning the number of lost notifications is directly tied to how long RabbitMQ was down. A longer or shorter outage would produce different results, making it difficult to generalize the findings to real-world failure scenarios where outage duration is unpredictable. In part 4b, the 100ms consumer delay was an arbitrary choice that may not reflect real-world conditions. A longer delay would produce a larger backlog, while a shorter delay may not produce meaningful backpressure at all, limiting the conclusions that can be drawn about RabbitMQ behavior under realistic consumer slowness.

Results

Correctness Verification Script After Stopping RabbitMQ

Check	Description	Result
1	No overselling	PASS - 33,031 confirmed orders
2	No duplicate order IDs	PASS - all 33,031 order IDs are unique
3	Confirmed orders exist in Orders DB	PASS - 33,031 confirmed orders found in DB
4	Inventory reconciliation	PASS - 33,031 confirmed + 66,969 remaining = 100,000 total tickets
5	Every confirmed order has a notification record	FAIL - 4,013 orders missing notifications

Check 5 of Correctness Verification Script After Artificial Delay

Time After Locust Stop	Confirmed Orders	Missing Notifications	Check 5 Result
Immediately	7,998	6,440	FAIL
3 minutes	7,998	3,076	FAIL
6 minutes	7,998	0	PASS

Analysis

The results from part 4a demonstrate that a RabbitMQ broker outage had no impact on the correctness of the purchase path. Checks 1 through 4 all passed, confirming that no overselling occurred, all order IDs were unique, and inventory was perfectly reconciled. However, Check 5 failed with 4,013 missing notifications, meaning every order confirmed during the broker downtime window lost its notification permanently. This directly supports the tradeoff identified in this experiment: by designing the Order Service to continue processing orders regardless of broker availability, the system preserved purchase correctness at the cost of at-most-once notification delivery.

The results from part 4b demonstrate that RabbitMQ successfully acted as a buffer between the Order Service and the Notification Service under backpressure. During the Locust run, the Order Service maintained 121.2 RPS with a median response time of 27ms, a p95 of 51ms, and zero failures, showing no degradation despite the Notification Service falling behind. Immediately after the Locust run stopped, 6,440 notifications were still unprocessed in the queue. The Notification Service continued draining the backlog after the load ended, reducing missing notifications to 3,076 at 3 minutes and reaching zero at 6 minutes, at which point all checks passed. This confirms that RabbitMQ held all messages safely during the period when the consumer could not keep up, and that the system eventually reached consistency once load subsided. The Order Service was never affected by the slow consumer, demonstrating that async messaging successfully decoupled producer throughput from consumer throughput.

As mentioned earlier, there are limitations to this analysis. The broker outage in part 4a was triggered manually and lasted a controlled duration, meaning the number of lost notifications would differ under a longer or shorter real-world outage. Similarly, the 100ms consumer delay in part 4b was an arbitrary choice, and a significantly longer delay may have produced different conclusions about queue stability.

Conclusions

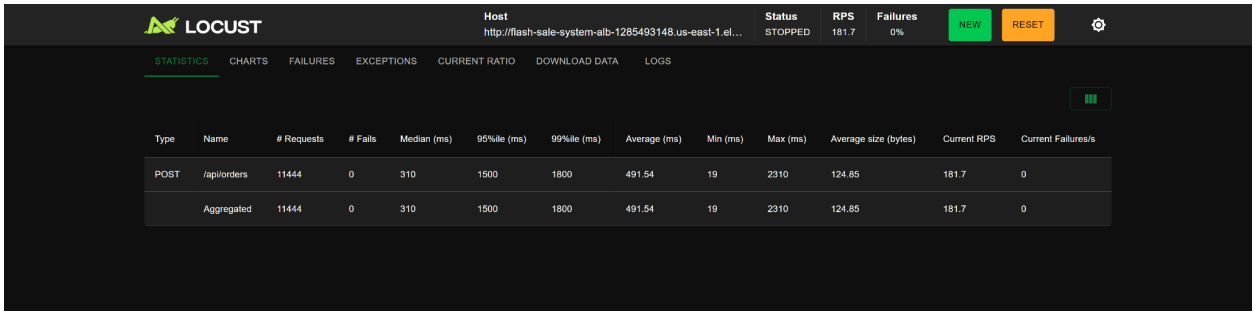
The results of this experiment reveal two important properties of the system's messaging layer. In part 4a, the broker outage had no impact on order correctness as inventory was perfectly reconciled and no duplicate orders were created. However, 4,013 notifications were

permanently lost during the downtime window. A more robust approach would be to write the pending notifications in the Orders database with the order and have a background process read from that table to publish to RabbitMQ with retries. This way, if RabbitMQ is down at the time of the order, the notification would not be lost and would be published once the broker recovers.

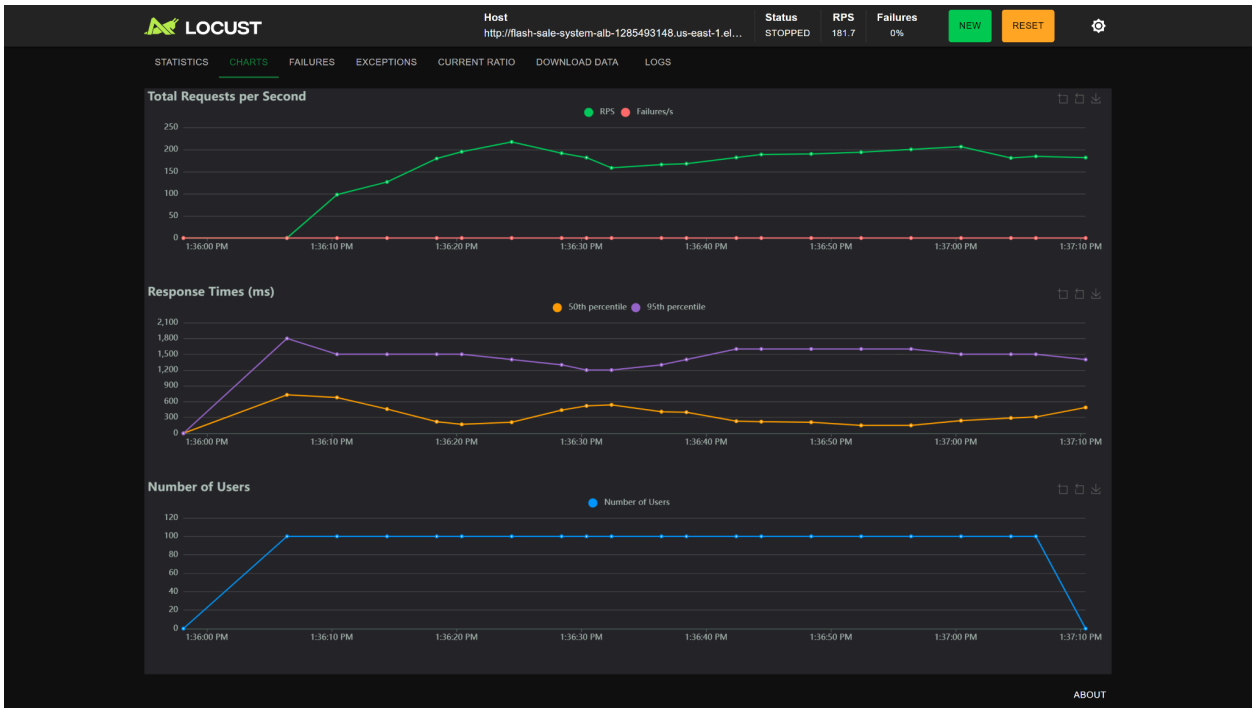
In part 4b, RabbitMQ successfully buffered all messages during the backpressure period and the system reached consistency within 6 minutes of load subsiding. This confirms that async messaging effectively decouples producer throughput from consumer throughput, allowing the Order Service to continue processing orders at full speed regardless of how fast the Notification Service can consume them.

Appendix

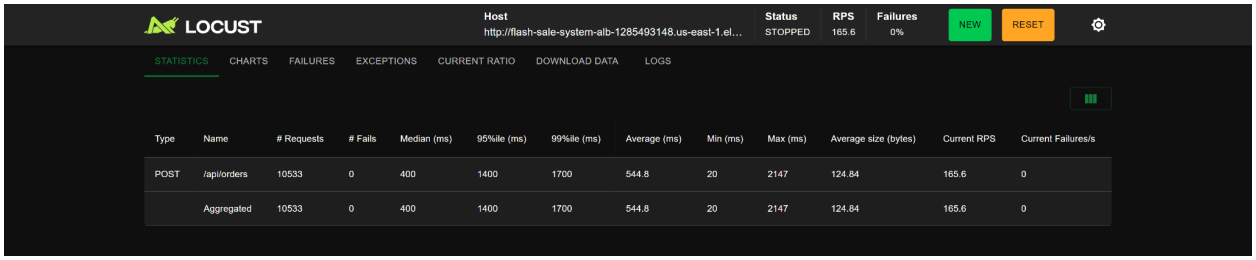
Experiment 1 Graphs and Charts



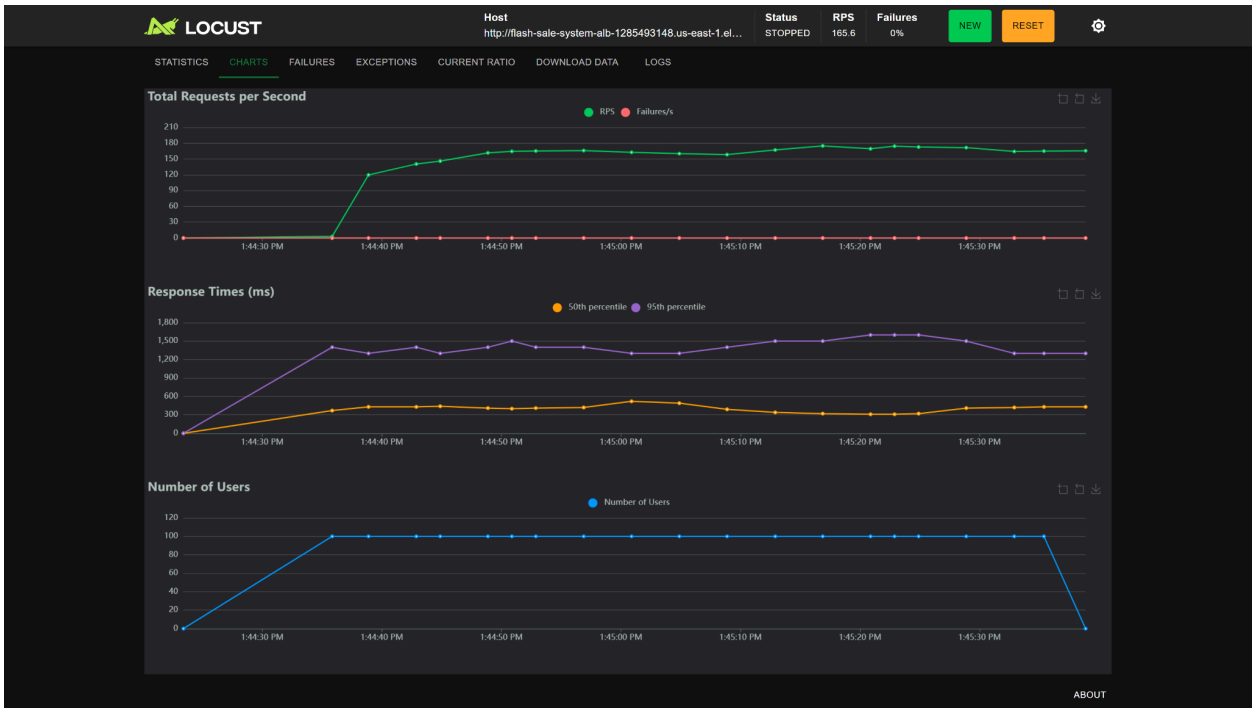
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/orders	11444	0	310	1500	1800	491.54	19	2310	124.85	181.7	0
	Aggregated	11444	0	310	1500	1800	491.54	19	2310	124.85	181.7	0



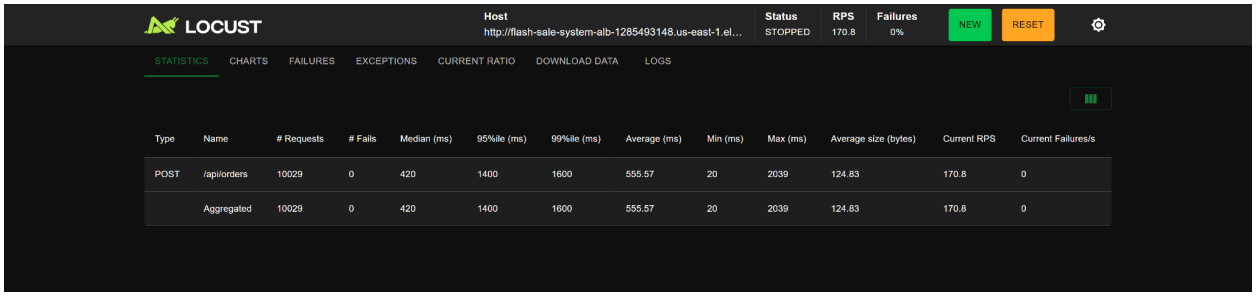
Experiment 1a - Locust Results for 1 Order Service ECS Task



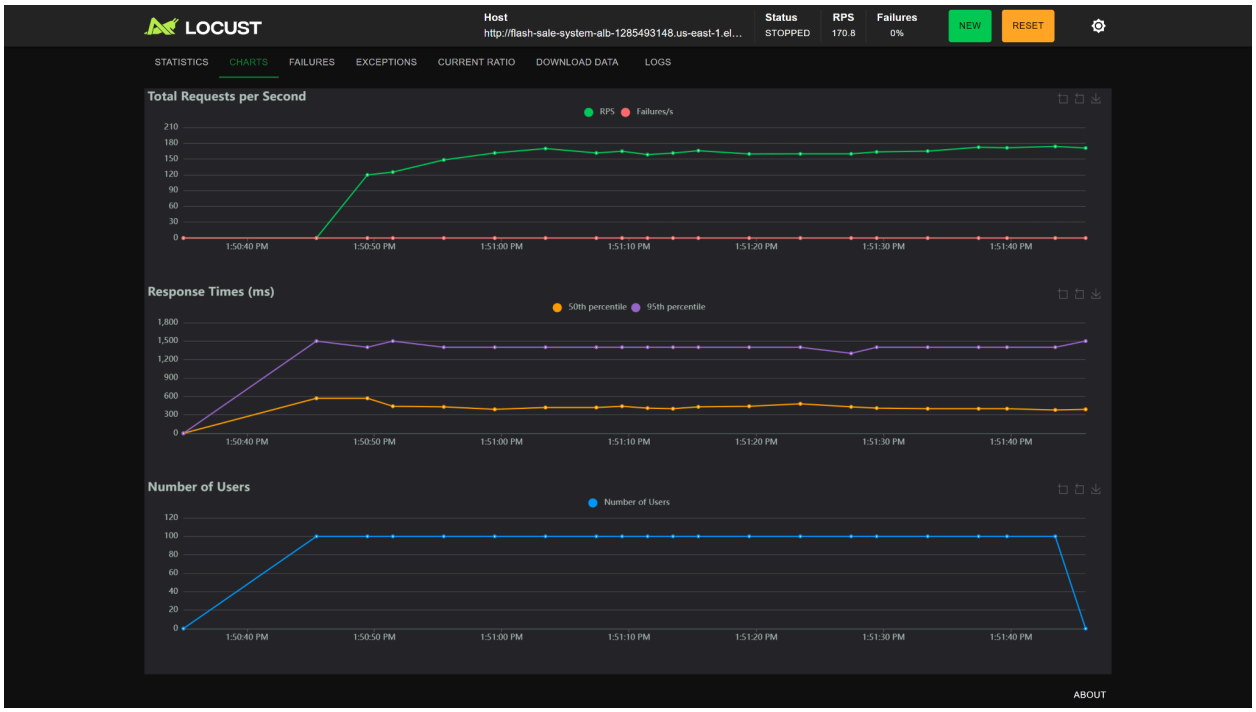
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/orders	10533	0	400	1400	1700	544.8	20	2147	124.84	165.6	0
	Aggregated	10533	0	400	1400	1700	544.8	20	2147	124.84	165.6	0



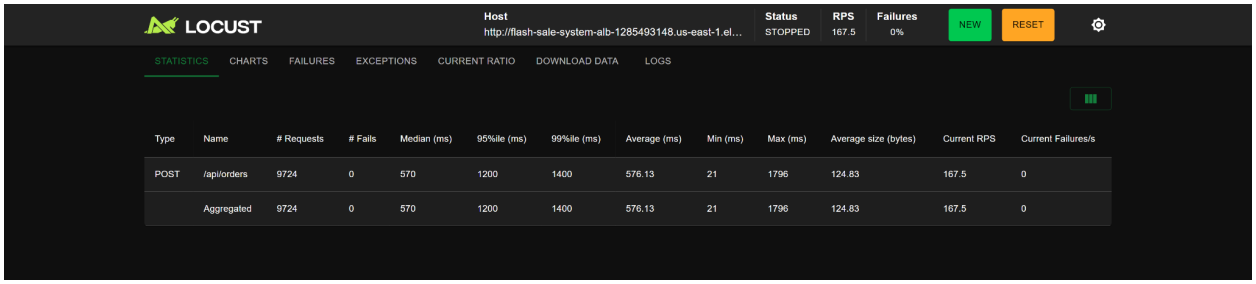
Experiment 1a - Locust Results for 2 Order Service ECS Tasks



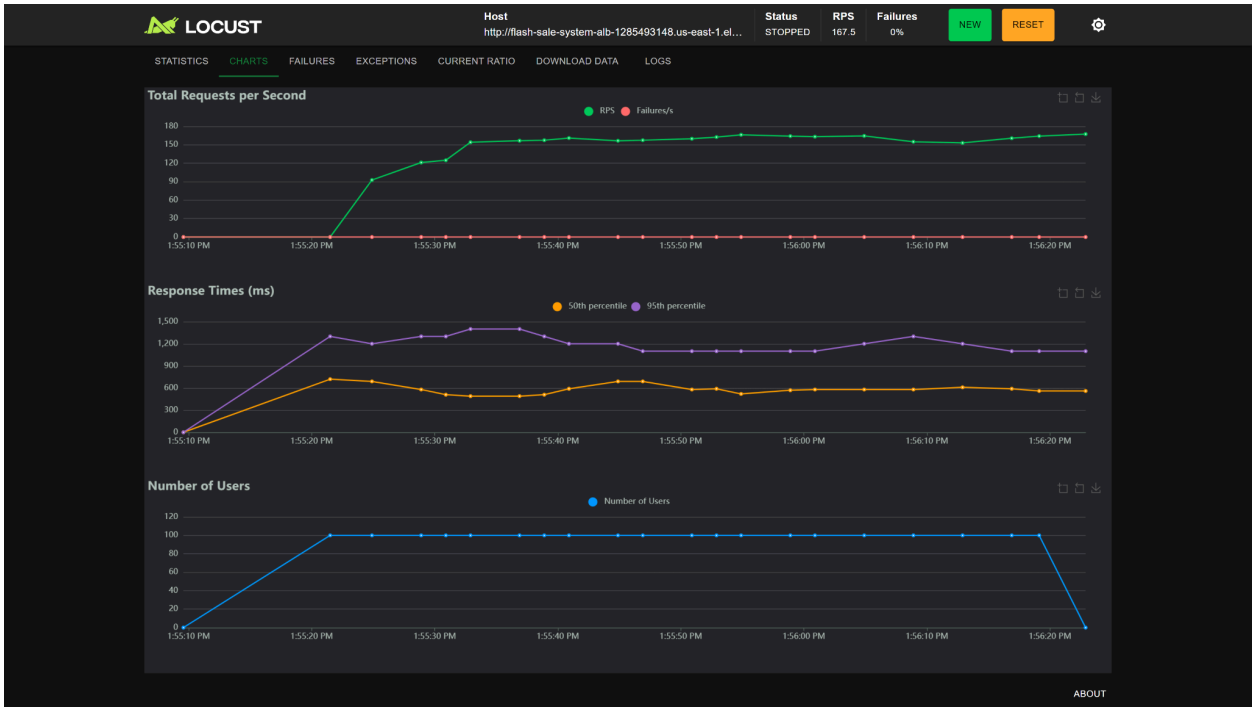
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/orders	10029	0	420	1400	1600	555.57	20	2039	124.83	170.8	0
	Aggregated	10029	0	420	1400	1600	555.57	20	2039	124.83	170.8	0



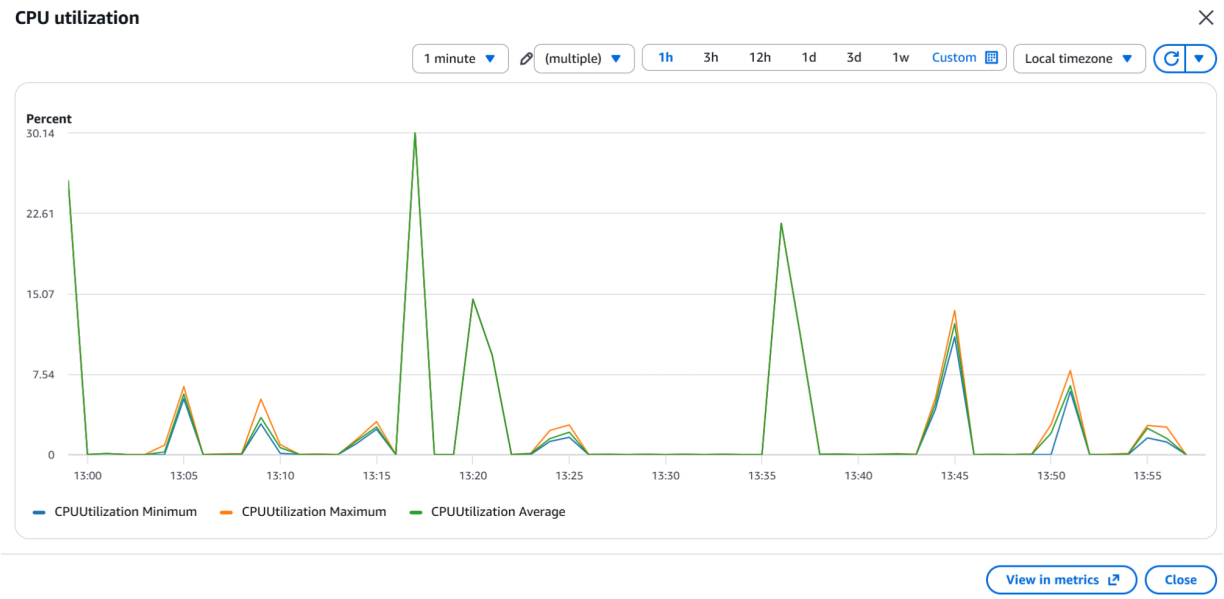
Experiment 1a - Locust Results for 4 Order Service ECS Tasks



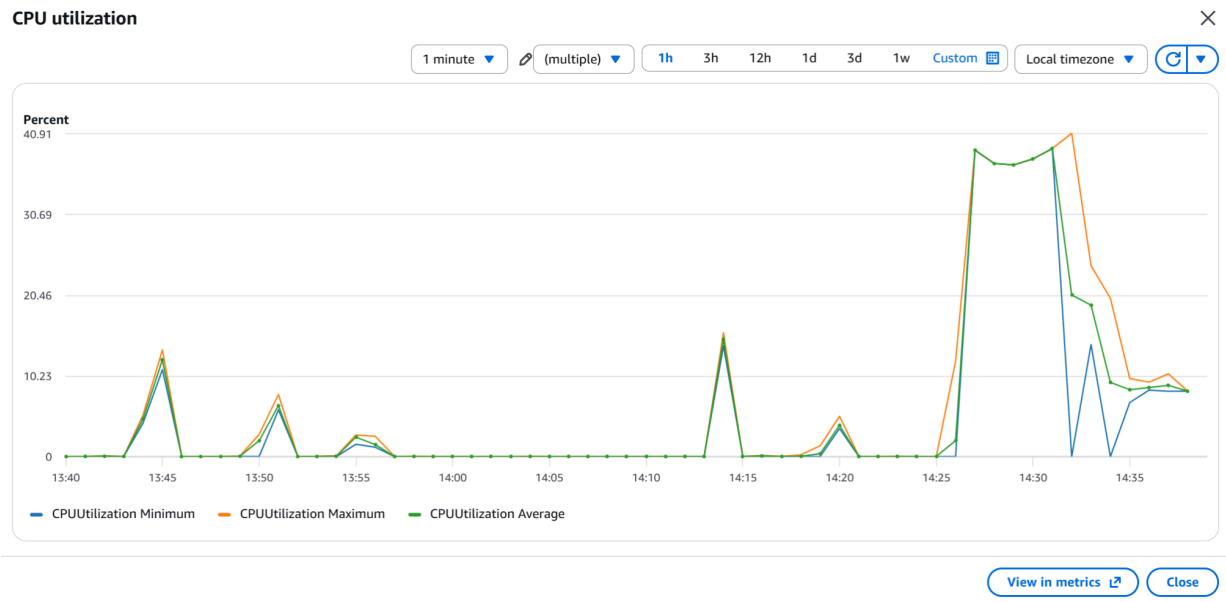
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/orders	9724	0	570	1200	1400	576.13	21	1796	124.83	167.5	0
	Aggregated	9724	0	570	1200	1400	576.13	21	1796	124.83	167.5	0



Experiment 1a - Locust Results for 8 Order Service ECS Tasks

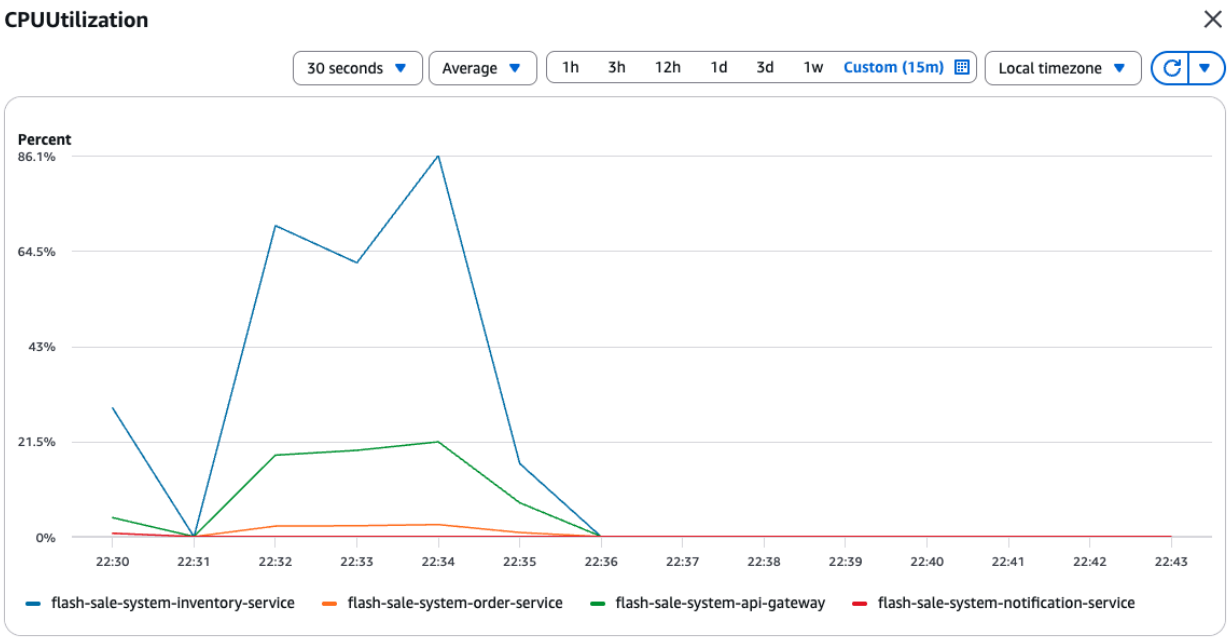


Experiment 1a - CPU Usage for ECS Tasks (starts at 13:35 for 1, 2, 4, 8 tasks respectively for each peak)



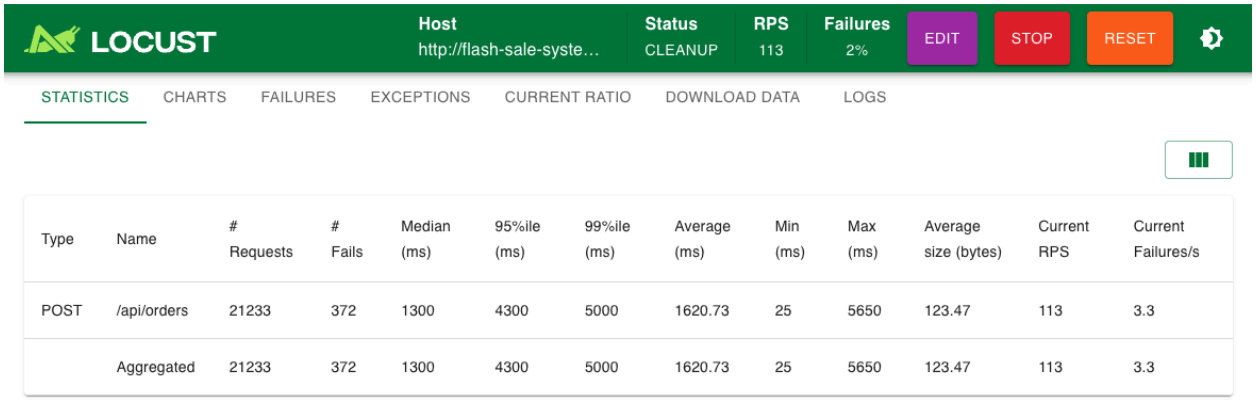
Experiment 1b - Autoscaling CPU Usage (starts at 14:25)

Experiment 2 Graphs and Charts



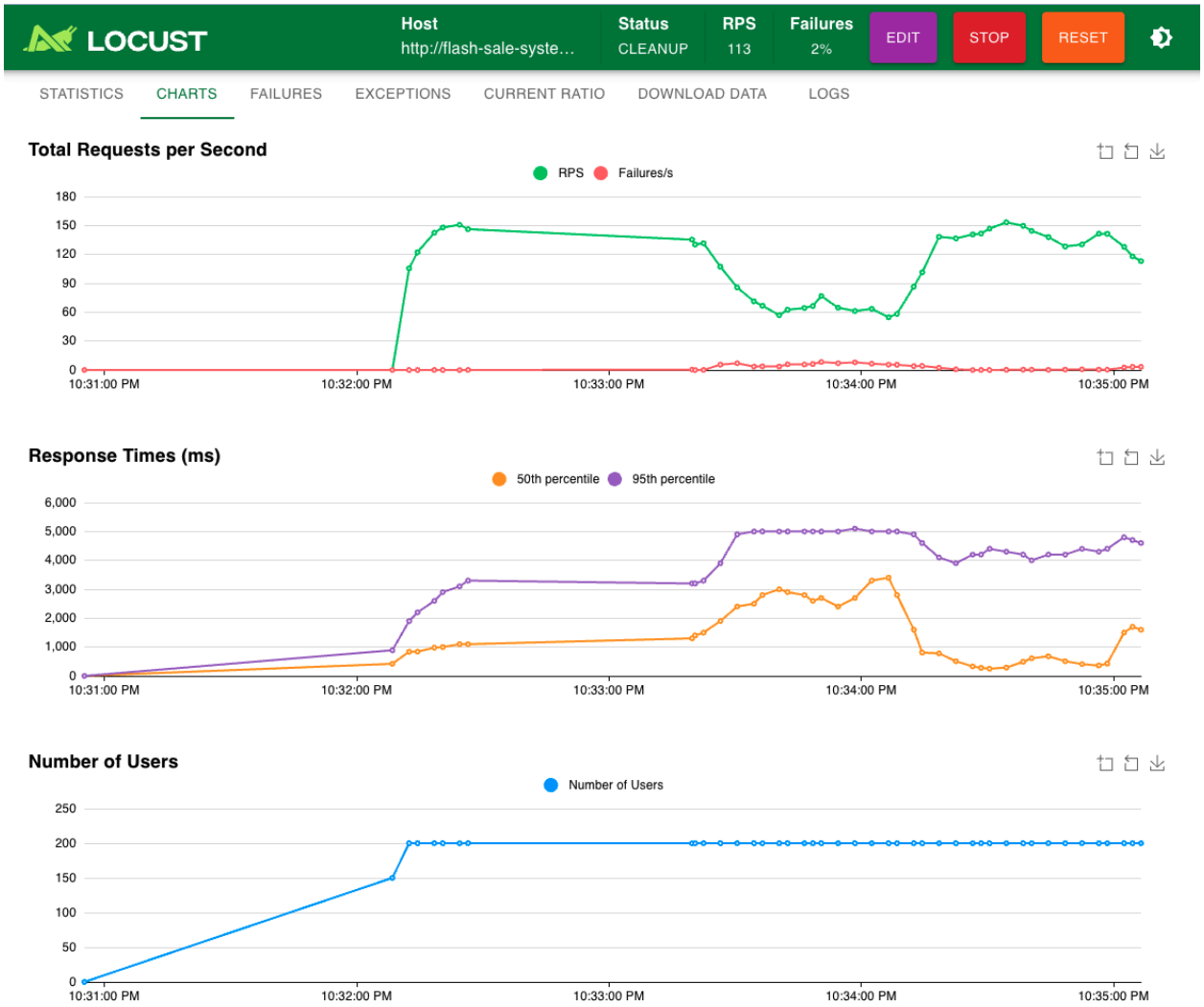
Experiment 2 - CPU Utilization for all Services

Experiment 2 - Request Count

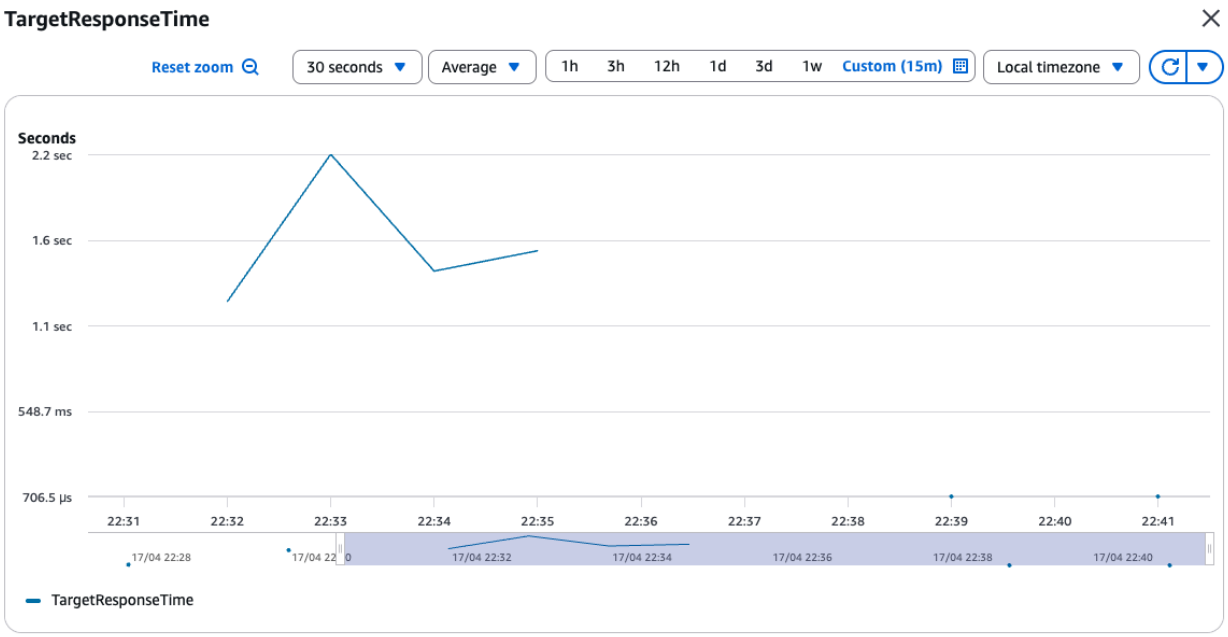


Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/orders	21233	372	1300	4300	5000	1620.73	25	5650	123.47	113	3.3
	Aggregated	21233	372	1300	4300	5000	1620.73	25	5650	123.47	113	3.3

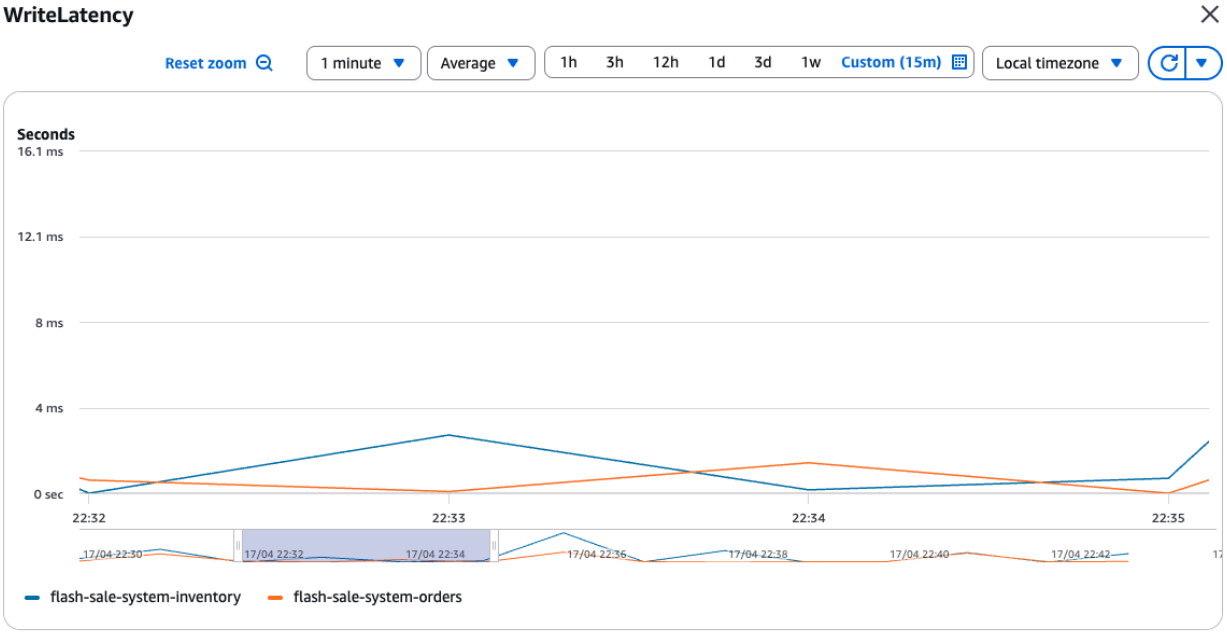
Experiment 2 - Locust Statistics



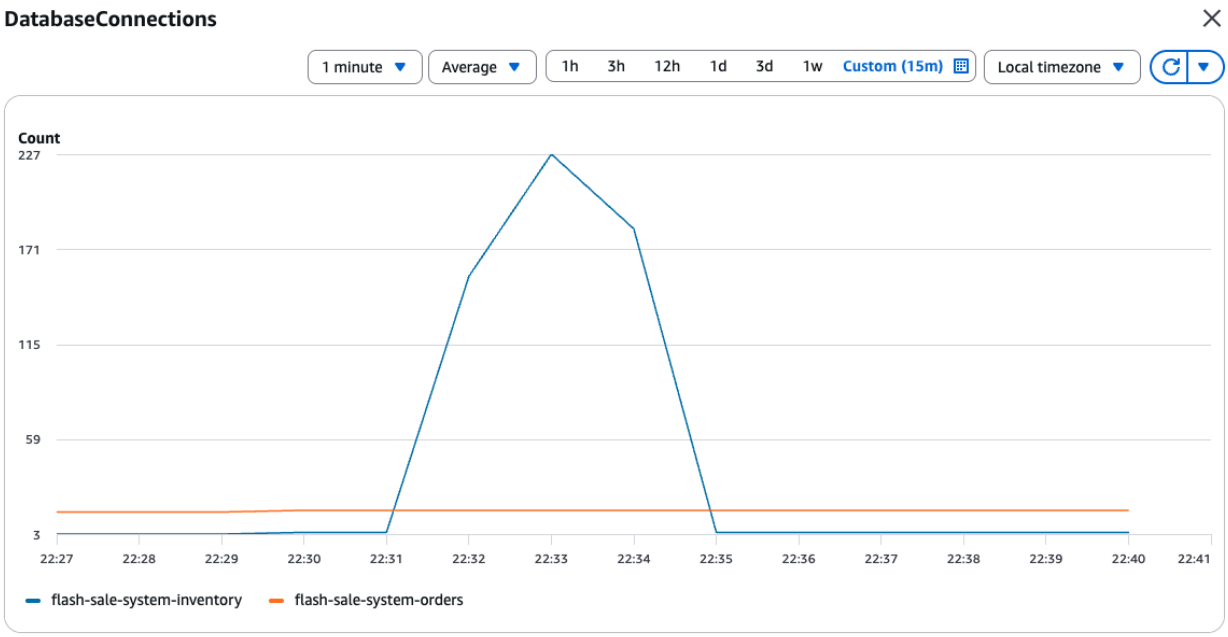
Experiment 2 - Locust Charts



Experiment 2 - Target Response Time



Experiment 2 - Write Latency for Inventory and Order DB

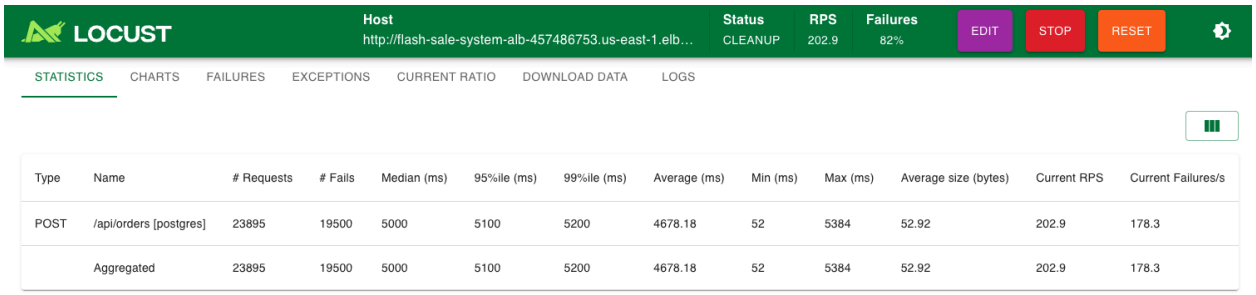


Experiment 2 - Database Connections for Inventory and Order DB

```
tonyhsu@Chia-hsiangs-Laptop flash-sale-system % /Users/tonyhsu/.pyenv/versions/3.12.5/bin
ash-sale-system/scripts/harness.py

Waiting 5s for RabbitMQ notification queue to drain...
  Done waiting

[Check 1] No overselling (DB confirmed orders)...
  PASS — 100 confirmed orders (limit 100)

[Check 2] No duplicate order IDs...
  PASS — all 100 order IDs are unique

[Check 3] Confirmed orders exist in orders DB...
  PASS — 100 confirmed orders found in DB

[Check 4] Inventory reconciliation (confirmed + remaining = initial)...
  PASS — 100 confirmed + 0 remaining = 100

[Check 5] Every confirmed order has a notification record...
  PASS — all confirmed orders have a notification record

────────────────────────────────────────
ALL CHECKS PASSED — system is correct
tonyhsu@Chia-hsiangs-Laptop flash-sale-system %
```

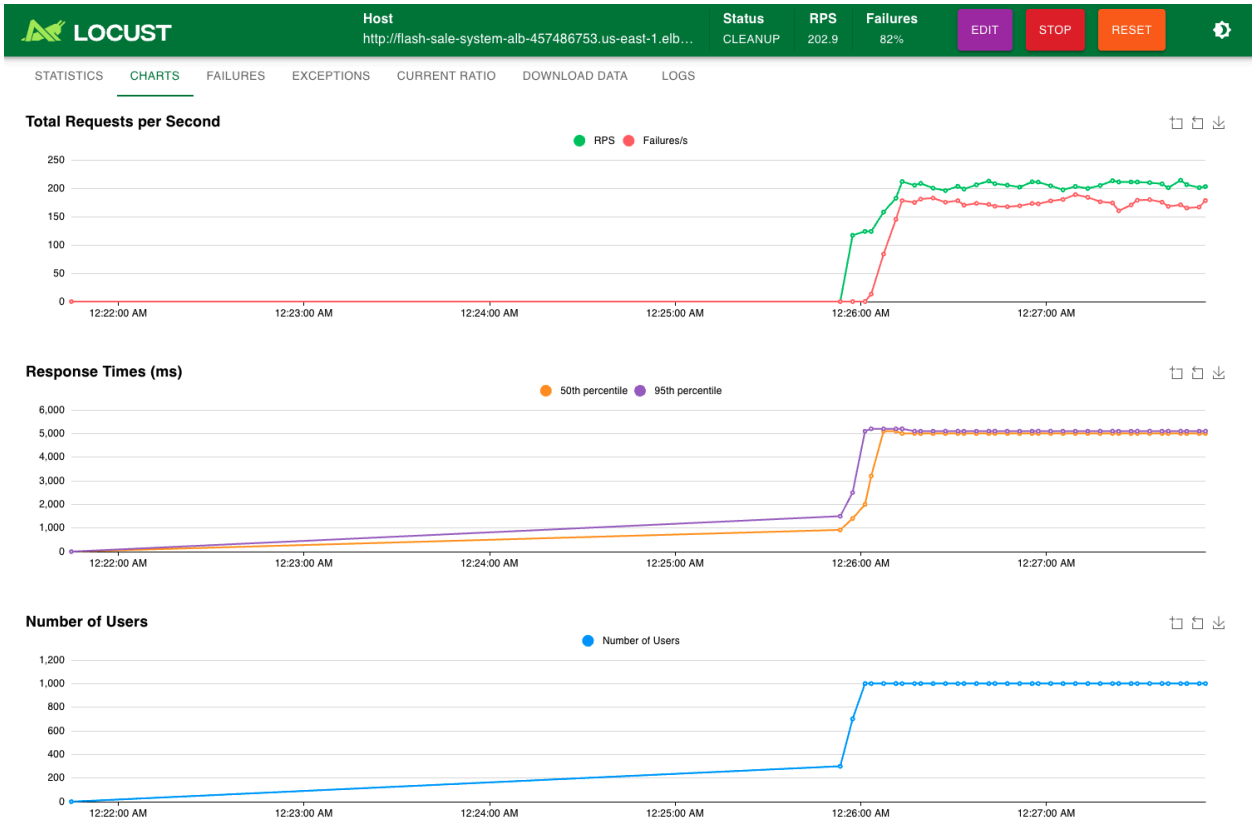
Experiment 2 - Consistency Verification Test Results

Experiment 3 Graphs and Charts

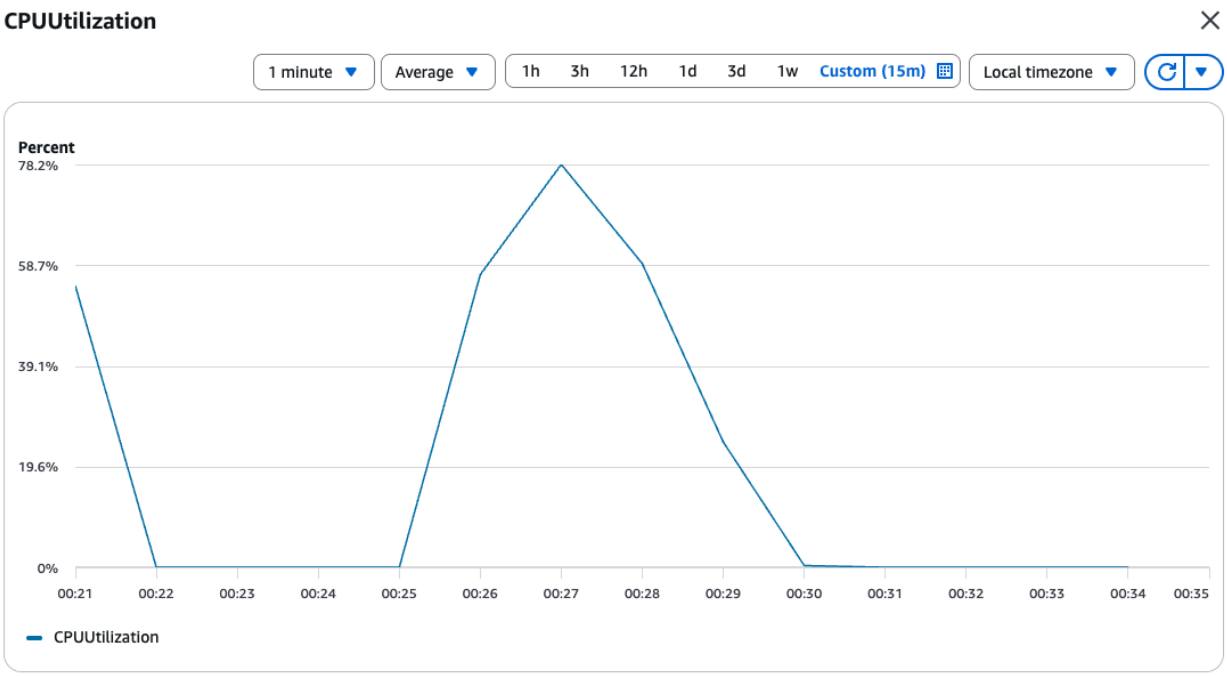


Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/orders [postgres]	23895	19500	5000	5100	5200	4678.18	52	5384	52.92	202.9	178.3
	Aggregated	23895	19500	5000	5100	5200	4678.18	52	5384	52.92	202.9	178.3

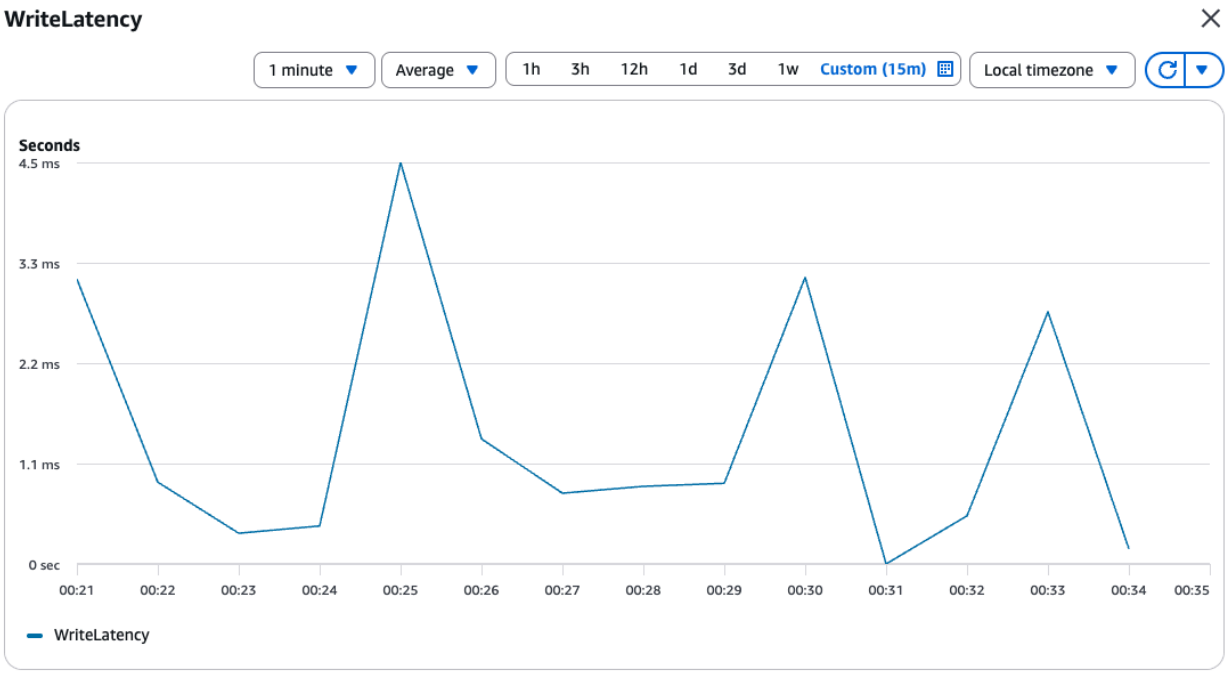
Experiment 3 - Postgres-only Locust Statistics



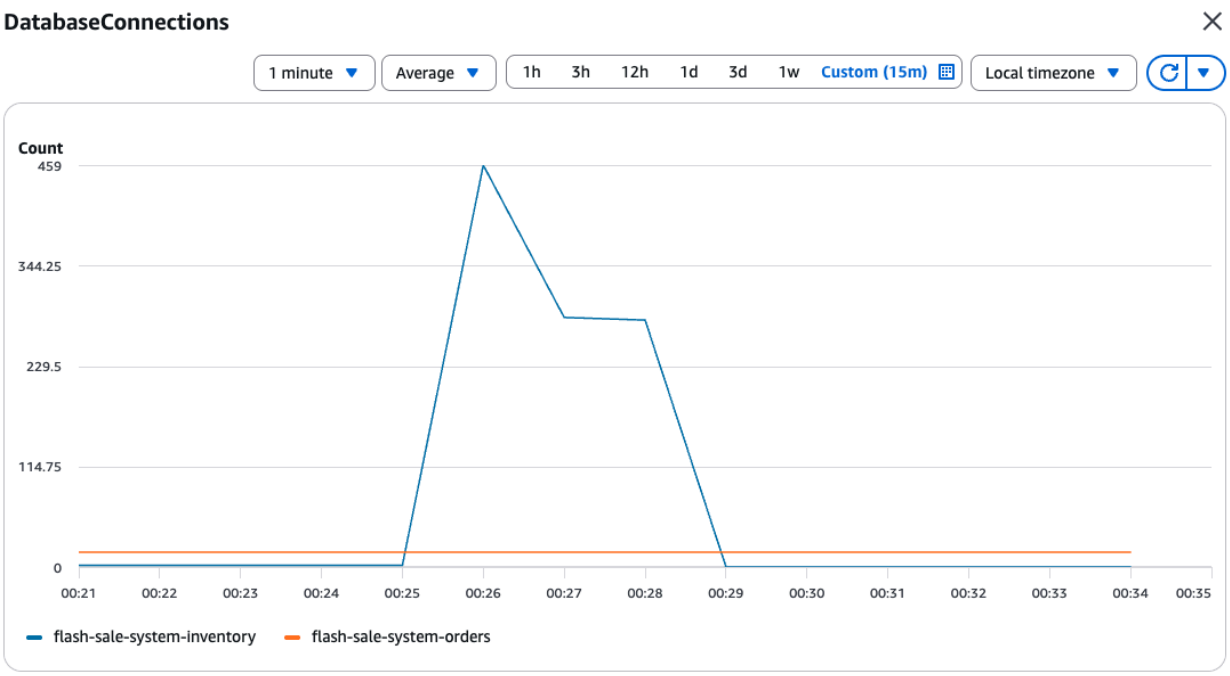
Experiment 3 - Postgres-only Locust Charts



Experiment 3 - Postgres-only CPU Utilization

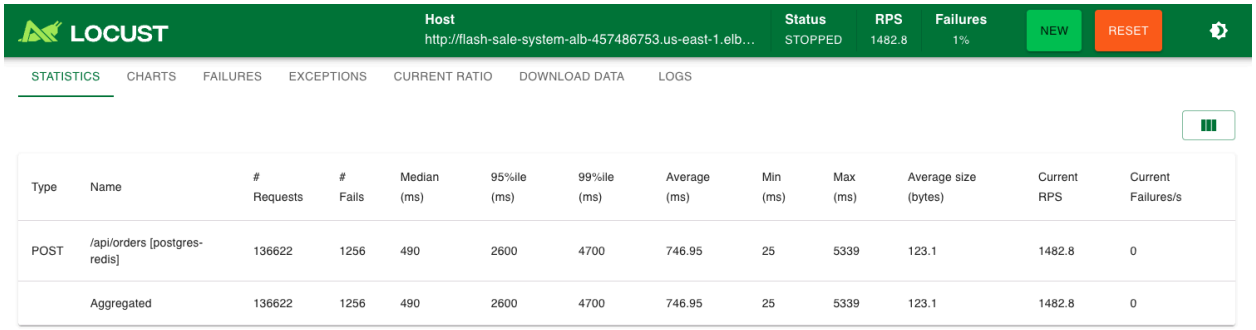


Experiment 3 - Postgres-only Write Latency



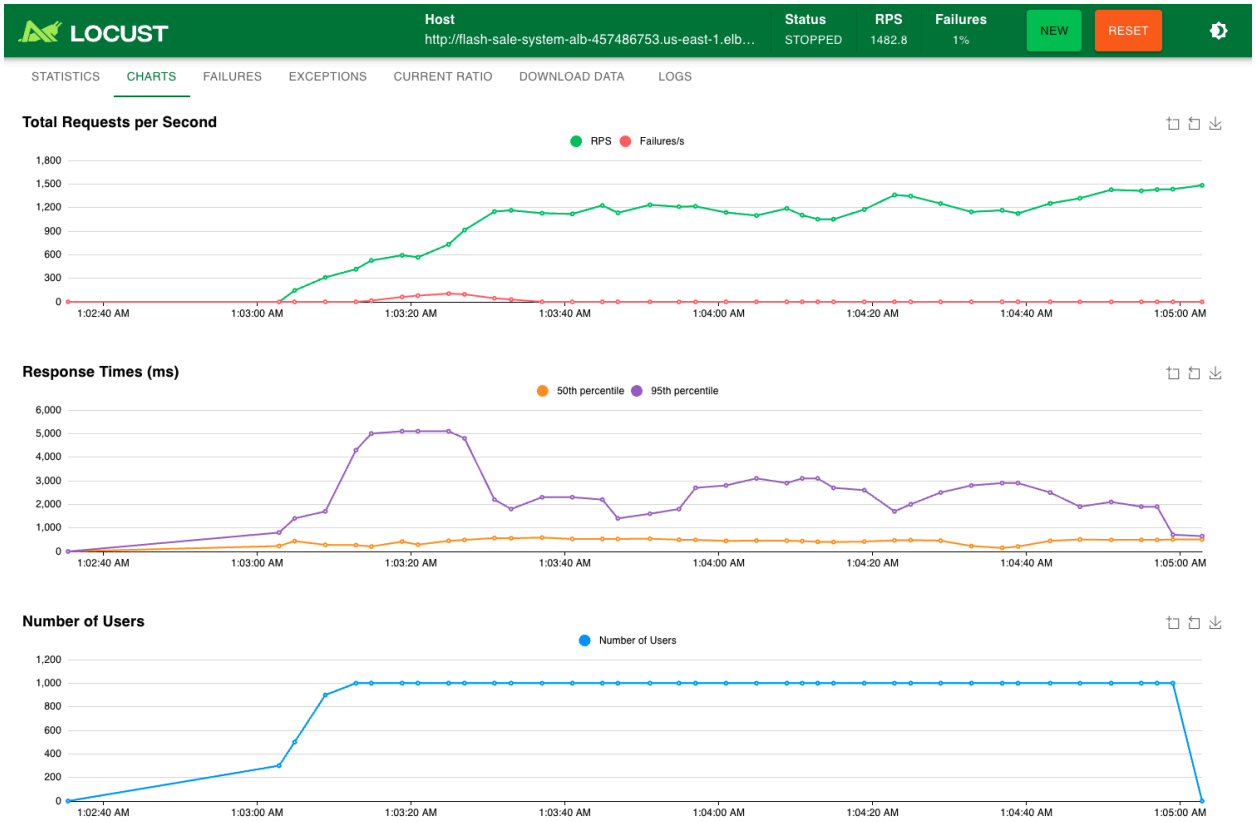
Experiment 3 - Postgres-Only Database Connections

Experiment 3 - Postgres-Only Correctness Verification Script

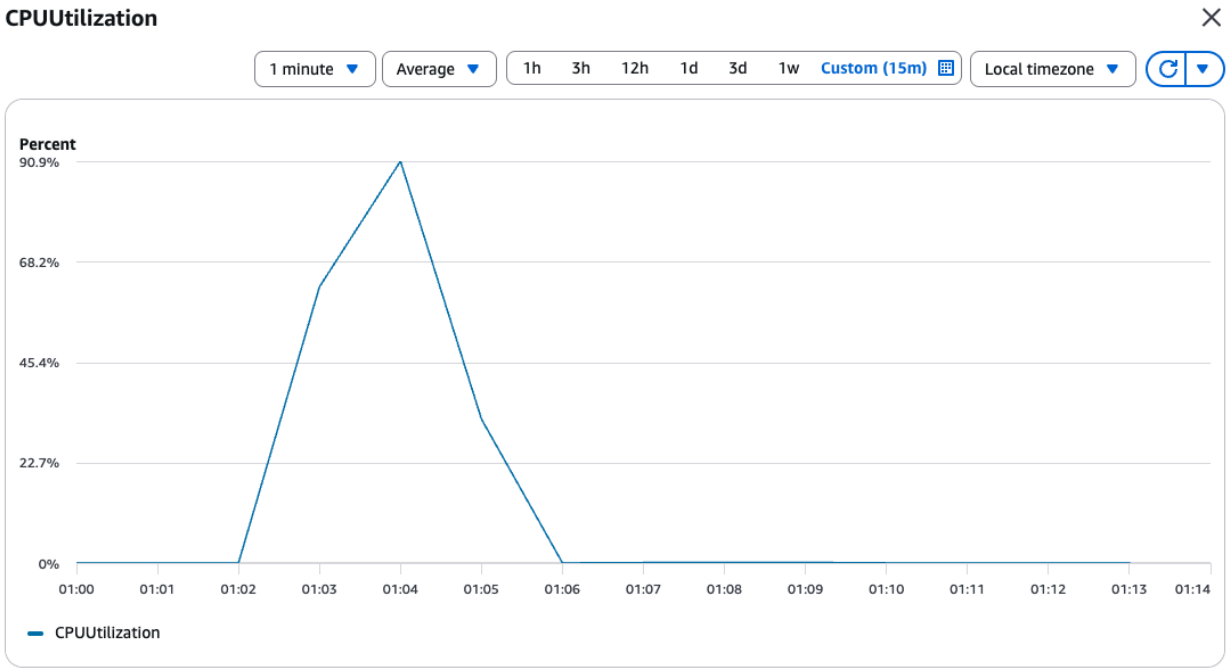


Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/orders [postgres-redis]	136622	1256	490	2600	4700	746.95	25	5339	123.1	1482.8	0
	Aggregated	136622	1256	490	2600	4700	746.95	25	5339	123.1	1482.8	0

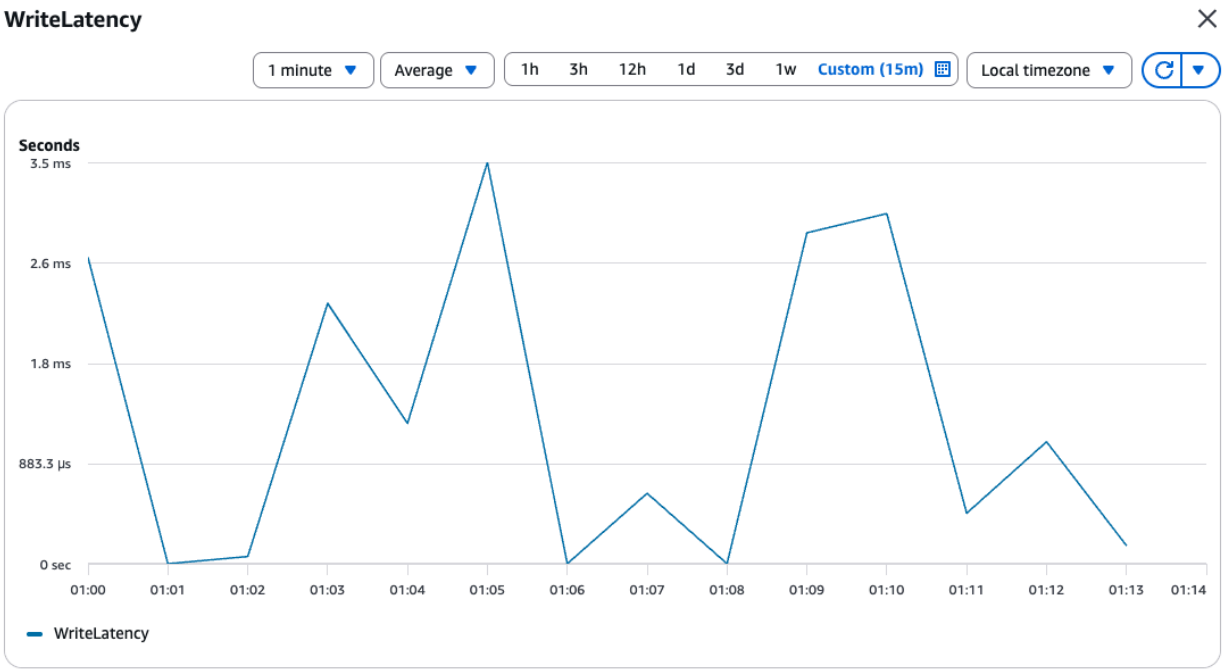
Experiment 3 - Redis-Postgres Locust Statistics



Experiment 3 - Redis-Postgres Locust Charts

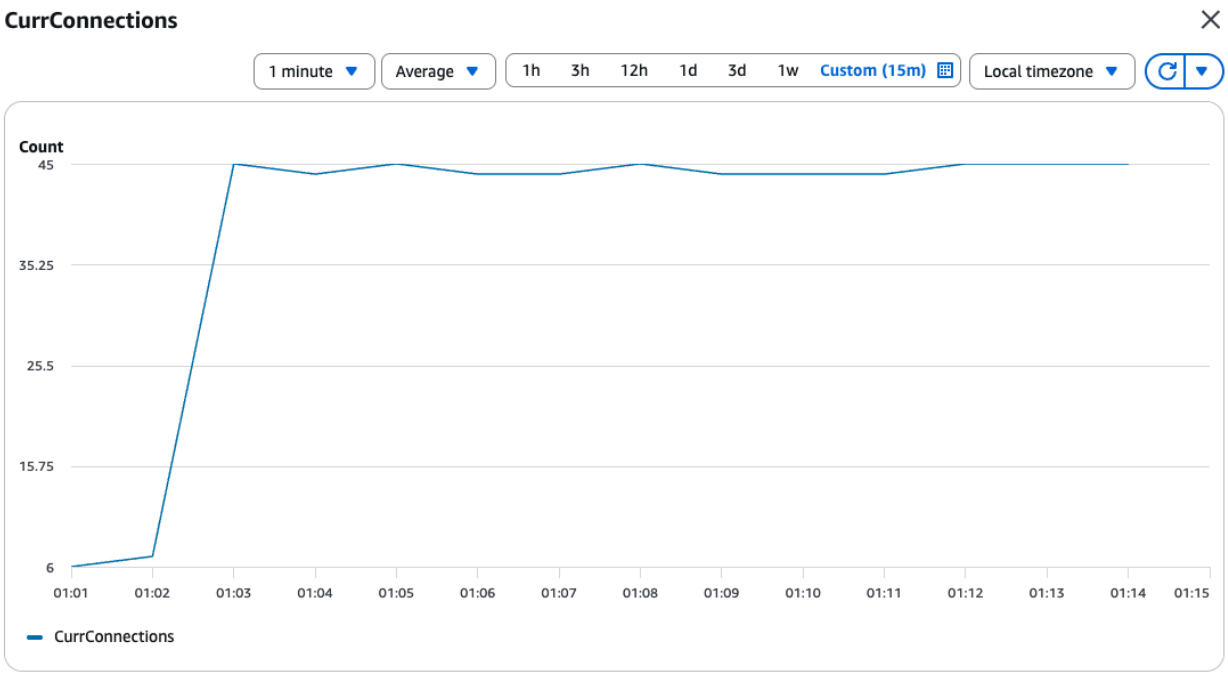


Experiment 3 - Redis-Postgres CPU Utilization



Experiment 3 - Redis-Postgres Write Latency

Experiment 3 - Redis-Postgres Database Connections



Experiment 3 - Redis-Postgres ElastiCache CurrConnections

```
tonyhsu@Chia-hsiangs-Laptop flash-sale-system % /Users/tonyhsu/.pyenv/versions/3.12.5/bin/python /Use

Waiting 5s for RabbitMQ notification queue to drain...
  Done waiting

[Check 1] No overselling (DB confirmed orders)...
  PASS — 10000 confirmed orders (limit 10000)

[Check 2] No duplicate order IDs...
  PASS — all 10000 order IDs are unique

[Check 3] Confirmed orders exist in orders DB...
  PASS — 10000 confirmed orders found in DB

[Check 4] Inventory reconciliation (confirmed + remaining = initial)...
  FAIL — 10000 confirmed + 1302 remaining = 11302 (expected 10000)

[Check 5] Every confirmed order has a notification record...
  PASS — all confirmed orders have a notification record

  ────────────────────────────────────────
SOME CHECKS FAILED — review output above
tonyhsu@Chia-hsiangs-Laptop flash-sale-system %
```

Experiment 3 - Redis-Postgres Correctness Verification Script

Experiment 4 Graphs and Charts

2026-04-19T05:06:31.277Z	[GIN] 2026/04/19 - 05:06:31 | 200 | 7.07ms | 10.0.11.136 | P…	ecs/order-service/ebe4e18d89494aff92997dc4178e15ec
2026-04-19T05:06:31.872Z	[GIN] 2026/04/19 - 05:06:31 | 200 | 5.07ms | 10.0.10.174 | P…	ecs/order-service/0c55524ea7724295bec26b66784ad179
2026-04-19T05:06:32.072Z	[GIN] 2026/04/19 - 05:06:32 | 200 | 5.07ms | 10.0.10.174 | P…	ecs/order-service/0c55524ea7724295bec26b66784ad179
2026-04-19T05:06:32.291Z	2026/04/19 05:06:32 WARNING: failed to publish notification …	ecs/order-service/0c55524ea7724295bec26b66784ad179
2026-04-19T05:06:32.339Z	2026/04/19 05:06:32 WARNING: failed to publish notification …	ecs/order-service/0c55524ea7724295bec26b66784ad179
2026-04-19T05:06:32.503Z	[GIN] 2026/04/19 - 05:06:32 | 200 | 5.07ms | 10.0.10.174 | P…	ecs/order-service/0c55524ea7724295bec26b66784ad179
2026-04-19T05:06:32.580Z	2026/04/19 05:06:32 WARNING: failed to publish notification …	ecs/order-service/0c55524ea7724295bec26b66784ad179

Experiment 4a - Lost Connection to RabbitMQ From Order Service

2026-04-19T05:06:32.234Z	2026/04/19 05:06:32 Notification recorded: order 790e6b95-fb…	ecs/notification-service/838329c0e81e436eb7b37c8437df7603
2026-04-19T05:06:32.236Z	2026/04/19 05:06:32 Notification recorded: order ea303ed7-ff…	ecs/notification-service/88cb0bc7fca54232abfadbd895c8989e
2026-04-19T05:06:32.237Z	2026/04/19 05:06:32 Notification recorded: order 87549d42-d2…	ecs/notification-service/838329c0e81e436eb7b37c8437df7603
2026-04-19T05:06:32.243Z	2026/04/19 05:06:32 RabbitMQ connection lost — reconnecting …	ecs/notification-service/88cb0bc7fca54232abfadbd895c8989e
2026-04-19T05:06:32.243Z	2026/04/19 05:06:32 RabbitMQ connection lost — reconnecting …	ecs/notification-service/838329c0e81e436eb7b37c8437df7603
2026-04-19T05:06:37.243Z	2026/04/19 05:06:37 Notification service: connecting to Rabb…	ecs/notification-service/838329c0e81e436eb7b37c8437df7603
2026-04-19T05:06:37.244Z	2026/04/19 05:06:37 Notification service: connecting to Rabb…	ecs/notification-service/88cb0bc7fca54232abfadbd895c8989e
2026-04-19T05:06:37.248Z	2026/04/19 05:06:37 Failed to connect to RabbitMQ: dial tcp …	ecs/notification-service/838329c0e81e436eb7b37c8437df7603
2026-04-19T05:06:37.249Z	2026/04/19 05:06:37 Failed to connect to RabbitMQ: dial tcp …	ecs/notification-service/88cb0bc7fca54232abfadbd895c8989e

Experiment 4a - Lost Connection to RabbitMQ From Notification Service

2026-04-19T05:07:06.673Z	2026/04/19 05:07:06 WARNING: failed to publish notification …	ecs/order-service/0c55524ea7724295bec26b66784ad179
2026-04-19T05:07:06.673Z	[GIN] 2026/04/19 - 05:07:06 | 200 | 6.53ms | 10.0.10.174 | P…	ecs/order-service/0c55524ea7724295bec26b66784ad179
2026-04-19T05:07:06.675Z	2026/04/19 05:07:06 WARNING: failed to publish notification …	ecs/order-service/0c55524ea7724295bec26b66784ad179
2026-04-19T05:07:06.675Z	[GIN] 2026/04/19 - 05:07:06 | 200 | 5.24ms | 10.0.11.136 | P…	ecs/order-service/ebe4e18d89494aff92997dc4178e15ec
2026-04-19T05:07:06.676Z	[GIN] 2026/04/19 - 05:07:06 | 200 | 5.58ms | 10.0.10.174 | P…	ecs/order-service/0c55524ea7724295bec26b66784ad179
2026-04-19T05:07:06.678Z	2026/04/19 05:07:06 RabbitMQ reconnected	ecs/order-service/0c55524ea7724295bec26b66784ad179
2026-04-19T05:07:06.683Z	[GIN] 2026/04/19 - 05:07:06 | 200 | 4.61ms | 10.0.10.174 | P…	ecs/order-service/0c55524ea7724295bec26b66784ad179
2026-04-19T05:07:06.702Z	[GIN] 2026/04/19 - 05:07:06 | 200 | 5.9ms | 10.0.11.136 | PO…	ecs/order-service/ebe4e18d89494aff92997dc4178e15ec
2026-04-19T05:07:06.715Z	[GIN] 2026/04/19 - 05:07:06 | 200 | 5.65ms | 10.0.11.136 | P…	ecs/order-service/ebe4e18d89494aff92997dc4178e15ec

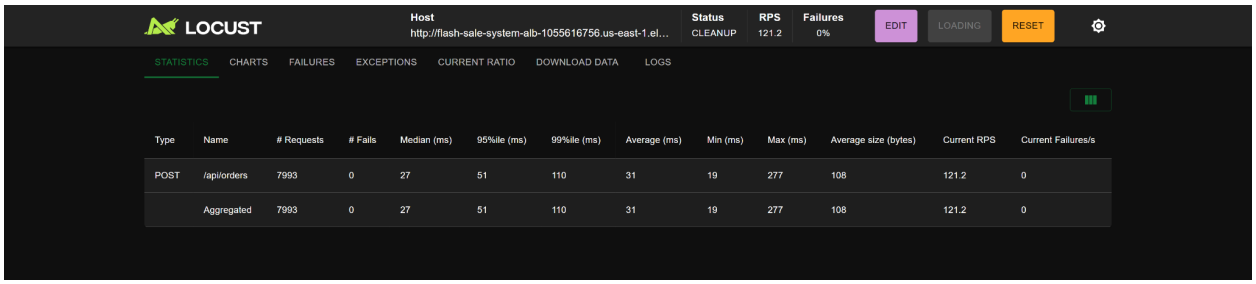
Experiment 4a - Reconnection to RabbitMQ From Order Service

2026-04-19T05:07:02.266Z	2026/04/19 05:07:02 Notification service: connecting to Rabb…	ecs/notification-service/88cb0bc7fca54232abfadbd895c8989e
2026-04-19T05:07:02.269Z	2026/04/19 05:07:02 Failed to connect to RabbitMQ: dial tcp …	ecs/notification-service/88cb0bc7fca54232abfadbd895c8989e
2026-04-19T05:07:02.277Z	2026/04/19 05:07:02 Notification service: connecting to Rabb…	ecs/notification-service/838329c0e81e436eb7b37c8437df7603
2026-04-19T05:07:02.278Z	2026/04/19 05:07:02 Failed to connect to RabbitMQ: dial tcp …	ecs/notification-service/838329c0e81e436eb7b37c8437df7603
2026-04-19T05:07:07.273Z	2026/04/19 05:07:07 Notification service: connecting to Rabb…	ecs/notification-service/88cb0bc7fca54232abfadbd895c8989e
2026-04-19T05:07:07.283Z	2026/04/19 05:07:07 Notification service: connecting to Rabb…	ecs/notification-service/838329c0e81e436eb7b37c8437df7603
2026-04-19T05:07:07.294Z	2026/04/19 05:07:07 Notification service: connected, waiting…	ecs/notification-service/88cb0bc7fca54232abfadbd895c8989e
2026-04-19T05:07:07.297Z	2026/04/19 05:07:07 Notification recorded: order 9ec04db6-cf…	ecs/notification-service/88cb0bc7fca54232abfadbd895c8989e
2026-04-19T05:07:07.299Z	2026/04/19 05:07:07 Notification recorded: order 62e259af-cd…	ecs/notification-service/88cb0bc7fca54232abfadbd895c8989e

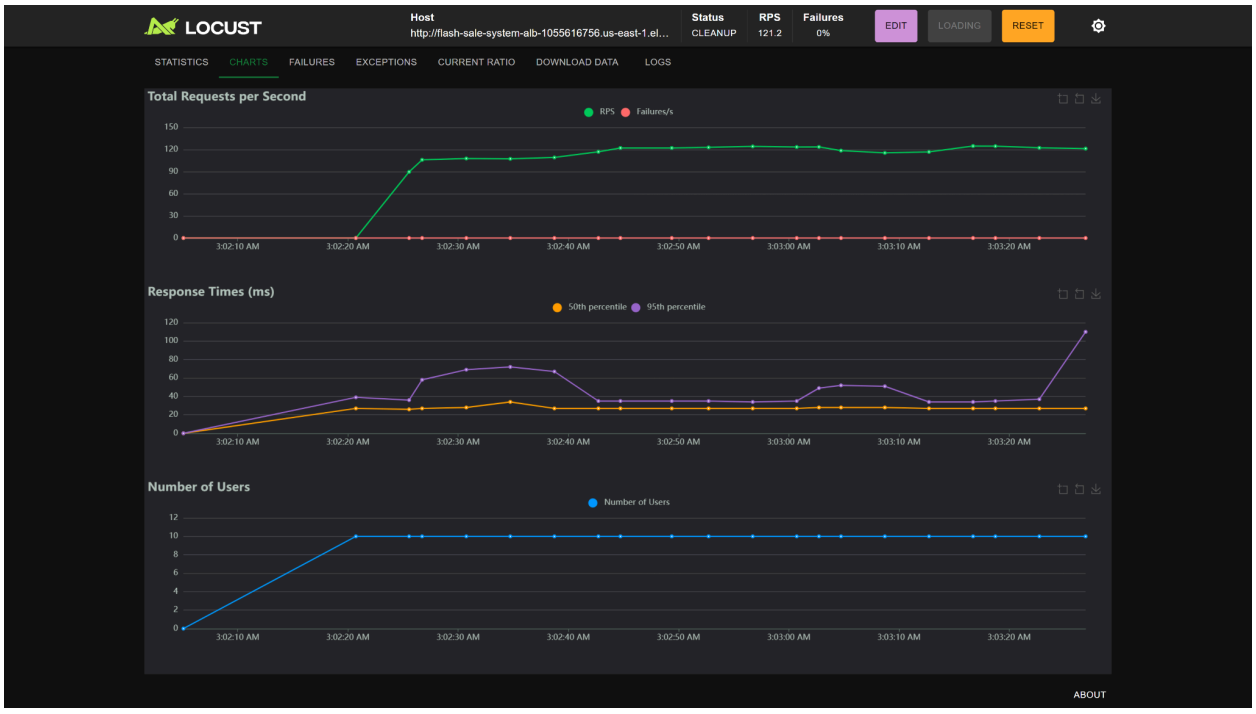
Experiment 4a - Reconnection to RabbitMQ From Notification Service

```
Waiting 5s for RabbitMQ notification queue to drain...
  Done waiting

[Check 1] No overselling (DB confirmed orders)...
  PASS — 33031 confirmed orders (limit 100000)

[Check 2] No duplicate order IDs...
  PASS — all 33031 order IDs are unique

[Check 3] Confirmed orders exist in orders DB...
  PASS — 33031 confirmed orders found in DB

[Check 4] Inventory reconciliation (confirmed + remaining = initial)...
  PASS — 33031 confirmed + 66969 remaining = 100000

[Check 5] Every confirmed order has a notification record...
  FAIL — 4013 orders missing notifications: ['3ffb79c4-5291-48f8-8a53-8480732b776b', '1da04241-3f6c-4772-8fb3-124bde282f52', 'ec10ef40-62c4-4907-b222-0122d2aa8745', '0f6a49f8-2e
05-49c2-8b6d-b25d7be13b7d', 'eaa389af-8687-4b3a-b706-3605daf5fceb', 'f920d118-d1e7-4ff6-8eb5-330f2be802bd', '80f16799-6a4a-4922-8949-5bcd1d37ad26', '00e76574-34dd-4ecb-8daf-5376
b2e6691b', '78676491-0f20-49e5-82c3-b3cec4af8990', 'a8625e1f-23cb-4555-8e2c-cedeff911ebe', '2774b8b2-87b7-418c-9680-a3f439643687', '0ff5d005-d449-4931-97b7-ab565b4acb03', 'e8ed5
eaf-75cc-4db0-8988-000c738c479e', 'b11e7799-4c9d-48e0-836f-a7e629eb30ac', '3c56e7e8-a337-49a4-a3db-29e11d9cc0a9', '01c45204-5ab5-4349-aac0-b7dff577b2df', '26c5ccf1-ffb0-4a81-9be
b-96f443baca7c', '86a4666d-8aff-426f-987f-22ea894beb2c', 'ca54032e-38cb-43e3-9df8-b39c033f5463', '3fab80be-0f86-422d-bd14-22662fd5bfff', 'b73fa53e-e05b-4477-b472-0e439cebdb63',
```

Experiment 4a - Correctness Verification Script



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/orders	7993	0	27	51	110	31	19	277	108	121.2	0
	Aggregated	7993	0	27	51	110	31	19	277	108	121.2	0



Experiment 4b - Locust Results

```
Waiting 5s for RabbitMQ notification queue to drain...
  Done waiting

[Check 1] No overselling (DB confirmed orders)...
  PASS — 7998 confirmed orders (limit 10000)

[Check 2] No duplicate order IDs...
  PASS — all 7998 order IDs are unique

[Check 3] Confirmed orders exist in orders DB...
  PASS — 7998 confirmed orders found in DB

[Check 4] Inventory reconciliation (confirmed + remaining = initial)...
  PASS — 7998 confirmed + 2002 remaining = 10000

[Check 5] Every confirmed order has a notification record...
  FAIL — 6440 orders missing notifications.

────────────────────────────────────────
SOME CHECKS FAILED — review output above
```

Experiment 4b - Correctness Verification Script Immediately After Stopping Load Test

```
Waiting 5s for RabbitMQ notification queue to drain...
  Done waiting

[Check 1] No overselling (DB confirmed orders)...
  PASS — 7998 confirmed orders (limit 10000)

[Check 2] No duplicate order IDs...
  PASS — all 7998 order IDs are unique

[Check 3] Confirmed orders exist in orders DB...
  PASS — 7998 confirmed orders found in DB

[Check 4] Inventory reconciliation (confirmed + remaining = initial)...
  PASS — 7998 confirmed + 2002 remaining = 10000

[Check 5] Every confirmed order has a notification record...
  FAIL — 3076 orders missing notifications.

────────────────────────────────────────
SOME CHECKS FAILED — review output above
```

Experiment 4b - Correctness Verification Script 3 Minutes After Stopping Load Test

```
Waiting 5s for RabbitMQ notification queue to drain...
  Done waiting

[Check 1] No overselling (DB confirmed orders)...
  PASS — 7998 confirmed orders (limit 10000)

[Check 2] No duplicate order IDs...
  PASS — all 7998 order IDs are unique

[Check 3] Confirmed orders exist in orders DB...
  PASS — 7998 confirmed orders found in DB

[Check 4] Inventory reconciliation (confirmed + remaining = initial)...
  PASS — 7998 confirmed + 2002 remaining = 10000

[Check 5] Every confirmed order has a notification record...
  PASS — all confirmed orders have a notification record

────────────────────────────────────────
ALL CHECKS PASSED — system is correct
```

Experiment 4b - Correctness Verification Script 6 Minutes After Stopping Load Test